

CLEBER JORGE AMARAL

**DESENVOLVIMENTO DE FIRMWARE E SOFTWARE
DE PROGRAMAÇÃO PARA UM CLP DE PLACA
ÚNICA, BASEADOS NA LINGUAGEM STEP5**

FLORIANÓPOLIS, 2007

**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE
SANTA CATARINA - UNIDADE DE FLORIANÓPOLIS**

**DEPARTAMENTO ACADÊMICO DE ELETRÔNICA
CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS
ELETRÔNICOS**

**DESENVOLVIMENTO DE FIRMWARE E SOFTWARE
DE PROGRAMAÇÃO PARA UM CLP DE PLACA
ÚNICA, BASEADOS NA LINGUAGEM STEP5**

Trabalho de Conclusão de Curso
apresentado ao Curso Superior de
Tecnologia em Sistemas Eletrônicos do
Centro Federal de Educação Tecnológica de
Santa Catarina como requisito parcial para
obtenção do grau de Tecnólogo em
Sistemas Eletrônicos.

Orientadores:

Fernando L. R. Mussoi, M. Eng.

Valdir Noll, Dr. Eng.

CLEBER JORGE AMARAL

FLORIANÓPOLIS, 2007

DESENVOLVIMENTO DE FIRMWARE E SOFTWARE DE PROGRAMAÇÃO PARA UM CLP DE PLACA ÚNICA, BASEADOS NA LINGUAGEM STEP5

CLEBER JORGE AMARAL

Este trabalho foi julgado adequado para obtenção do Título de Tecnólogo em Sistemas Digitais e aprovado na sua forma final pela banca examinadora do Curso Superior de Tecnologia em Sistemas Digitais do Centro Federal de Educação Tecnológica de Santa Catarina.

Orientador: Fernando Luiz Rosa Mussoi, M. Eng.
DAELN – Depto. Acadêmico de Eletrônica do CEFET/SC

Orientador: Valdir Noll, Dr. Eng.
DAMM – Depto. Acadêmico de Metal-Mecânica do CEFET/SC

Banca examinadora:

Fernando Luiz Rosa Mussoi, M. Eng.

Valdir Noll, Dr. Eng.

Marco Valério Miorim Villaça, Dr. Eng.

Everton Luiz F. dos Santos, Dr. Eng.

DEDICATÓRIA

*Dedico este trabalho a minha família
e a minha gatinha.*

AGRADECIMENTOS

Aos orientadores Fernando Mussoi e Valdir Noll por acreditarem e abraçarem meu projeto.

À professora Cláudia R. Silveira e ao amigo Michael Silva pela ajuda na monografia.

Aos amigos que me apoiaram durante todo o curso: Jerônimo Lopes, Leonardo G. Pinheiro e Tiago Pereira.

À empresa Automatiza por investir em minha formação e acreditar em meu potencial.

De forma especial ao guru e amigo Eduardo Guse por sua paciência e dedicação neste e em outros tantos projetos.

RESUMO

Este trabalho tem como objetivo o desenvolvimento de um Controlador Lógico Programável, em placa única, capaz de realizar funções básicas. Baseia-se em padrões já estabelecidos no mercado a fim de facilitar a adaptação de usuários que já utilizam este tipo de dispositivo. Para o desenvolvimento do CLP foi criado um *software* para computador IBM-PC, capaz de traduzir a lista de instruções introduzida pelo usuário. Além disso foi criado um *firmware* (gravado no microcontrolador) que é responsável pela interpretação dos comandos e atuação. A linguagem STEP5, criada pela empresa Siemens, foi utilizada como base para o desenvolvimento do referido *firmware* e como linguagem de programação para o CLP. O desempenho do sistema alcançou as expectativas permitindo carregar um programa no CLP e executar diversas funções básicas, a partir de uma linguagem concordante com a norma IEC 61131.

Palavras-chave: Controlador Lógico Programável, STEP5, firmware, software.

ABSTRACT

This work objective is the development of a Programmable Logic Controller, in a single-board, capable to perform basic functions. It is based on standards already established at the market in order to facilitate adaptation of users that already use this device type. For the development of PLC, a software was created for IBM-PC computer, capable to translate the list of instructions introduced by the user. Besides that it was created a firmware (stored in the microcontroller) that is responsible for the interpretation of the commands and operation. The language STEP5, created by the Siemens company, was used as a base for the development of the referred firmware and as programming language for PLC. The performance of the system reached the hopes allowing to load a program in the PLC and to execute several basic functions, starting from a language that is accordant with the norm IEC 61131.

Key-words: Programmable Logic Controller, STEP5, firmware, software.

LISTA DE FIGURAS

FIGURA 1 - Diagrama de blocos de um CLP.....	21
FIGURA 2 - Frame open PLC (Triangle Research Inc., Pte. Ltd.).....	23
FIGURA 3 - CLP em gabinete individual (IDEC Corp.).....	24
FIGURA 4 - CLP modular (Omron Eletronics).....	24
FIGURA 5 - Variações quanto ao tamanho.....	25
FIGURA 6 - Seqüência de operação.....	28
FIGURA 7 - Exemplo de um programa Ladder.....	31
FIGURA 8 - Exemplo de uso do FBD.....	32
FIGURA 9 - Exemplo de uma lista de instruções.....	33
FIGURA 10 - Exemplo de código ST.....	34
FIGURA 11 - Exemplo de diagrama de funções seqüenciais.....	35
FIGURA 12 - Composição da mensagem MODBUS.....	37
FIGURA 13 - Diagrama de caso de uso.....	41
FIGURA 14 - Fotografia da placa.....	42
FIGURA 15 - Formato da instrução.....	46
FIGURA 16 - Exemplo de programa usando temporizador.....	50
FIGURA 17 - Exemplo de uso do contador.....	52
FIGURA 18 - Exemplo de uso de comparações.....	53
FIGURA 19 - Exemplos de constantes.....	54
FIGURA 20 - Diagrama de classes do firmware.....	56
FIGURA 21 - Software montador.....	58
FIGURA 22 - Diagrama de classes do software.....	59

FIGURA 23 - Diagrama de atividades da montagem.....	60
FIGURA 24 - Resposta de erro retornada pelo software.....	61
FIGURA 25 - Processo de carregamento de comandos no CLP.....	62
FIGURA 26 - Conexões da placa do CLP para o ensaio do elevador.....	66
FIGURA 27 - Esquema elétrico de acionamento do motor.....	67
FIGURA 28 - Trecho de código do elevador (subida).....	68
FIGURA 29 - Trecho de código do elevador (parada).....	69

LISTA DE TABELAS

TABELA 1 - Partes da norma IEC 61131.....	27
TABELA 2 - Estrutura de camadas do modelo OSI e camadas do MODBUS.....	38
TABELA 3 - Ocupação da memória RAM.....	44
TABELA 4 - Configuração da memória serial EEPROM.....	45
TABELA 5 - Exemplo de ocupação da memória EEPROM.....	46
TABELA 6 - Filtro de entrada.....	57
TABELA 7 - Estimativa de tempo do scan.....	65
TABELA 8 - Etapas do trabalho.....	72
TABELA 9 - Comparativo entre o CLP deste projeto e outros disponíveis no mercado	75

LISTA DE ABREVIATURAS E SIGLAS

BIT – *Binary Digit*, dígito binário, a menor unidade de informação binária (INDUSTRIAL, 2002).

bps – Bits por Segundo.

BYTE – Sequência de bits, grupo de bits processados em conjunto. (GARDINY, 2003).

CLP – Controlador Lógico programável ou PLC (*Programmable Logic Controller*)

CPU - *Central Processing Unit*, unidade central de processamento.

EEPROM ou E2PROM – Electrically Erasable and Programmable Read Only Memory, memória somente para leitura apagável e programável eletronicamente.

FBD – *Function Block Programming*, diagrama de blocos funcionais.

I/O – *In/Out*, Entrada/Saída.

IEC - *Internation Electrotechnical Comission*

IHM – Interface Homem-Máquina.

IL – *Instruction List*, lista de instruções.

GM – General Motors

Grafcet – GRAPhe Fonctionnel de Commande Étape Transition (BRYAN, 1997).

LD – *Ladder Diagram*, diagrama *ladder*.

LED – *Light Emiting Diode*, diodo emissor de luz.

OSI – Open Systems Interconnection

PC – *Personal Computer*, computador pessoal.

PCI – Placa de Circuito Impresso.

PID – *Proportional-Integral-Derivative Interface*, Interface Proporcional-Derival-Integrativo.

RAM – Random Access Memory, memória de acesso aleatório.

RLO – *Result ou a Logic Operation*, resultado de uma operação lógica.

SFC – *Sequential Function Charts*, diagrama de funções seqüenciais.

ST – Structured Text, texto estruturado.

SUMÁRIO

RESUMO.....	V
ABSTRACT.....	VI
LISTA DE FIGURAS.....	VII
LISTA DE TABELAS.....	IX
LISTA DE ABREVIATURAS E SIGLAS.....	X
1 INTRODUÇÃO.....	15
1.1 Justificativa.....	16
1.2 Definição do problema.....	16
2 OBJETIVOS.....	18
2.1 Objetivo Geral.....	18
2.2 Objetivos Específicos.....	18
3 REVISÃO DE LITERATURA.....	19
3.1 Definição.....	19
3.2 Resumo histórico.....	21
3.3 Classificação quanto ao gabinete.....	22
3.3.1 <i>Frame open</i>	22
3.3.2 <i>Gabinete Simples</i>	23
3.3.3 <i>Modular</i>	24
3.4 Classificação quanto ao tamanho.....	24
3.5 Norma IEC 61131.....	27
3.6 Seqüência de operação.....	28
3.7 Linguagens de programação.....	29
3.7.1 <i>Diagrama Ladder (LD)</i>	30
3.7.2 <i>Diagrama de Blocos Funcionais (FBD)</i>	31
3.7.3 <i>Lista de Instruções (IL)</i>	32
3.7.4 <i>Texto Estruturado (ST)</i>	33
3.7.5 <i>Diagrama de Funções Sequenciais (SFC)</i>	34
3.8 Protocolos de comunicação.....	35
3.8.1 <i>O protocolo MODBUS</i>	36

3.8.2 O protocolo MODBUS e o modelo OSI.....	38
4 PROJETO DO SOFTWARE E FIRMWARE DO CLP.....	40
4.1 Requisitos do projeto.....	40
4.2 Configuração do hardware.....	41
4.2.1 A placa.....	42
4.2.2 Configuração da memória volátil.....	43
4.2.3 Configuração da memória de aplicação.....	44
4.3 Conjunto de instruções.....	46
4.3.1 Lógicas de bits.....	48
4.3.2 Atribuições.....	49
4.3.3 Temporizadores.....	50
4.3.4 Contadores.....	51
4.3.5 Instruções de Comparação.....	52
4.3.6 Instrução de carga.....	53
4.3.7 Instruções de controle do programa e outras instruções.....	54
4.4 O firmware.....	55
4.5 Software montador.....	57
4.5.1 Processo de montagem.....	59
4.6 Carregamento dos comandos no CLP.....	61
4.6.1 Comunicação Serial.....	63
4.7 Aferição e ensaios.....	64
4.7.1 Estimativas de tempo.....	64
4.7.2 Exemplo de aplicação.....	66
5 METODOLOGIA.....	70
5.1 Métodos Aplicados.....	70
5.2 Apresentação, análise e discussão dos resultados.....	73
6 CONSIDERAÇÕES FINAIS.....	76
7 RECOMENDAÇÕES PARA TRABALHOS FUTUROS.....	77
BIBLIOGRAFIA COMPLEMENTAR.....	78
REFERÊNCIAS.....	80
APÊNDICES.....	83
APÊNDICE A – Estudo das instruções.....	84

APÊNDICE B – Esquemático da placa.....	85
APÊNDICE C – Programa do elevador.....	87
ANEXOS.....	88
ANEXO A – Códigos de funções públicas.....	89
ANEXO B – Conjunto completo de instruções STEP5.....	90
ANEXO C – Características do microcontrolador AT89C51ED2.....	91

1 INTRODUÇÃO

A indústria de equipamentos eletrônicos Automatiza tem freqüente necessidade de adaptação de seus produtos para atender seus clientes da área de segurança. Em cada instalação há um conjunto de informações distintas, tais como: número de usuários, área, exigências de segurança (seja para facilitar a evacuação ou contra invasão, evasão e contaminação). Os diversos sistemas de segurança como alarmes, controles de acesso, sensores de fumaça, sistemas de iluminação e acionamento muitas vezes precisam estar interligados.

O conceito “Controlador Lógico Programável” (CLP) veio a ser a tradução da solução para diversas necessidades geradas no dia-a-dia pelos clientes da Automatiza. Os diversos cenários trazem diferentes requisitos de produto, muitas vezes um produto, mesmo com características fixas e bem definidas, se ajusta à aplicação. Porém, em muitas outras situações se faz necessário o desenvolvimento de um dispositivo personalizado. O CLP pode ser programado e reprogramado de acordo com a necessidade no próprio local onde está instalado, por esta razão se adapta bem a diferentes requisitos de segurança.

Para desenvolver o CLP serão elaborados um *software* e um *hardware*. Através do *software* o usuário poderá digitar comandos que podem ser montados¹ e enviados ao *hardware* que interpreta e executa ciclicamente. Para este projeto são pesquisados os padrões e dispositivos já disponíveis no mercado a fim de minimizar impactos de adaptação ao novo produto por parte dos usuários, além de seguir as tendências e esforços da comunidade que busca a padronização dos dispositivos.

O *software* e o *firmware* serão implementados para suportarem a linguagem STEP5, criada pela empresa Siemens. A escolha desta linguagem se dá pelo fato de ser muito difundida entre usuários, por ter disponível bastante documentação e, principalmente, por estar de acordo com a norma IEC 61131.

Este documento procurará inicialmente dar embasamento aos conceitos e experiências associados ao projeto realizado. Portanto, após serem apresentados os objetivos deste trabalho, serão explanados conceitos importantes relacionados aos

¹ A montagem é o processo de tradução de mnemônicos booleanos em códigos binários que serão interpretados pelo *firmware*.

CLP's, como: definição, configurações típicas, um resumo histórico, introdução à norma IEC 61131, sequência de operação básica e introdução às linguagens de programação.

Em seguida, o projeto começará a ser descrito, primeiramente mostrando como houve a concepção, qual é a configuração do *hardware*, quais instruções foram definidas, como se dá a comunicação entre o dispositivo e o computador e, em seguida, será apresentado o *software* montador. Também serão explicados detalhes sobre a programação do *firmware*², limitações e características de construção do CLP.

1.1 Justificativa

Uma indústria de equipamentos eletrônicos encomendou o projeto para que possa comercializá-lo como um CLP de baixo custo e principalmente para que possa aplicá-lo em seus projetos de implantação e como evolução de seus produtos. Hoje há uma grande demanda de projetos que requerem desenvolvimento de placas específicas de circuito impresso. Por se adaptar a diversas situações e conter diversas entradas e saídas de uso geral será possível minimizar esta necessidade utilizando o CLP.

Os CLP's podem contribuir largamente na melhoria da qualidade e produtividade nos processos industriais, bem como atender a uma extensa gama de aplicações em diversas outras áreas. As principais qualidades que justificam o empenho em seu desenvolvimento são: reduzido espaço físico, flexibilidade de mudanças na lógica de controle, alta confiabilidade, possibilidade de expansão de entradas e saídas, lógicas similares para diferentes fabricantes e possibilidade de comunicação com computadores.

1.2 Definição do problema

“A formulação do problema prende-se ao tema proposto: ela esclarece a

2 O *firmware* é o programa que está gravado no microcontrolador.

dificuldade específica com a qual se defronta e que se pretende resolver por intermédio da pesquisa” (LAKATOS; MARCONI, 2003, p.220).

Neste projeto o problema se resume em desenvolver um *software* para computador IBM-PC e um *firmware* para ser gravado no microcontrolador de uma placa de circuito impresso já existente. Essa placa possui, principalmente, 16 entradas digitais, 8 relés de saída e uma porta de comunicação RS485.

O *software* para computador deve ser capaz de traduzir e enviar para a placa do CLP comandos que serão digitados em uma linguagem baseada na linguagem STEP5, desenvolvida pela empresa Siemens, grande fabricante de CLP's. O *firmware* deve ser capaz de receber os comandos do computador, gravá-los em uma memória do tipo não volátil e finalmente interpretá-los.

A interpretação dos comandos realizada pelo *firmware* deve ser de forma cíclica. As instruções suportadas serão as mais básicas que reúnem operações booleanas, contadores, temporizadores e comparações de números. Deve-se seguir os padrões sugeridos pela norma IEC 61131 e tendências adotadas pelos principais CLP's já disponíveis no mercado.

2 OBJETIVOS

2.1 Objetivo Geral

Desenvolver um *software* de programação e um *firmware* para realizar funções básicas de um CLP baseado nos padrões já existentes no mercado.

2.2 Objetivos Específicos

- Implementar um controlador lógico programável em placa única, utilizando padrões de mercado, a fim de facilitar a adesão de novos usuários ao novo produto proposto.
- Adaptar ao produto comandos de uma linguagem do tipo *Instruction List*³ (lista de instruções) já utilizada no mercado.
- Possibilitar programação via *software* de computador, desenvolvendo um programa montador. O usuário deve ter disponível uma interface onde poderá digitar comandos na linguagem STEP5 e gerar o arquivo binário que será carregado no produto via porta serial.
- Desenvolver o *firmware* do CLP de maneira que este seja definitivo e único em todas as aplicações.
- Permitir programação e reprogramação através do programa aplicação⁴ que deve ser gravado numa área de memória isolada e acessível ao usuário.
- Possibilitar programação do CLP através de instruções de lógicas de bits, atribuição de valores, de contagens, temporizações, além de instruções de carga e comparação de números.
- Permitir, futuramente, a programação através da conexão de um teclado externo e expansão de entradas e saídas.

³ Uma das 5 linguagens definidas pela norma IEC 61131-3.

⁴ O programa aplicação é desenvolvido pelo próprio usuário para resolver um problema específico.

3 REVISÃO DE LITERATURA

O conceito Controlador Lógico Programável é bastante amplo, suas capacidades e funções variam entre fabricantes e modelos. Há unidades criadas para tarefas básicas simplesmente substituindo um quadro de relés, mas há aplicações muito complexas que exigem alto desempenho e condição de resolução matemática de ponto flutuante (integrais e derivadas, por exemplo).

3.1 Definição

O termo Controlador Lógico Programável, ou simplesmente CLP, é a tradução de *Programmable Logic Controller* (PLC). É um dispositivo de controle de estado sólido que pode ser programado para controlar processos ou máquinas. Consiste em cinco componentes básicos: processador, memória, entradas e saídas, fonte de alimentação e um dispositivo de programação (INDUSTRIAL, 2002). O CLP é capaz de armazenar instruções que são executadas de maneira cíclica para realizar funções de controle tais como: sequências lógicas, temporizações, contagens, manipulação de dados, entre outras.

O dispositivo, assim como um computador, necessita receber um conjunto de comandos para executar tarefas; o conjunto de comandos é chamado programa aplicação. Cada instrução processada é considerada um passo do programa aplicação.

Diferentes CLP's podem desempenhar diferentes funções, algumas básicas, outras sofisticadas e específicas. Alguns exemplos são: lógicas booleanas, temporizações, contagens, operações aritméticas, manipulação de dados, controle PID e operações com ponto flutuante. Algumas características também destacam determinados modelos como: controles analógicos, comunicação em rede, recursos de monitoramento de execução do programa e diagnósticos, alta velocidade de varredura, elevada capacidade de corrente, comunicação com computadores, comunicação com interface homem-máquina.

Os principais blocos que compõem um CLP podem ser definidos, já suas funções e capacidades especificamente dependem do fabricante e do objetivo para

o qual ele foi construído. O número de entradas e saídas e a quantidade de memória de aplicação variam e normalmente são expansíveis. De acordo com Hackworth (2003, p.2-3), podemos observar os seguintes blocos:

1. Unidade Central de Processamento: pode ser um microprocessador ou microcontrolador que possua memória e circuitos auxiliares de controle. Neste bloco está a inteligência do CLP, há um circuito eletrônico que interpreta os códigos escritos pelo usuário.
2. Entradas e saídas digitais e/ou analógicas: as saídas podem ser, por exemplo, relés com contatos normalmente abertos ou normalmente fechados, podem haver entradas para uso específico como, por exemplo, para medição de corrente ou algum sensor especial.
3. Fonte de alimentação: responsável por fornecer energia ao CLP e, por vezes, a algum sistema externo.
4. Rack: proporciona conexões mecânicas e elétricas dos componentes supra citados. Alguns modelos de CLP's são montados numa estrutura modular que permite manipular suas capacidades de acordo com a aplicação e necessidade. Nem todos os modelos de CLP's são construídos na forma de rack.
5. Unidade de programação: interface homem-máquina que permite ao projetista criar ou editar o programa que será executado. Pode ser um teclado e visor dedicado, um *handheld*⁵ ou computador com *software*.

A FIGURA 1 mostra um diagrama de blocos dos principais componentes de um CLP.

5 Computador de mão

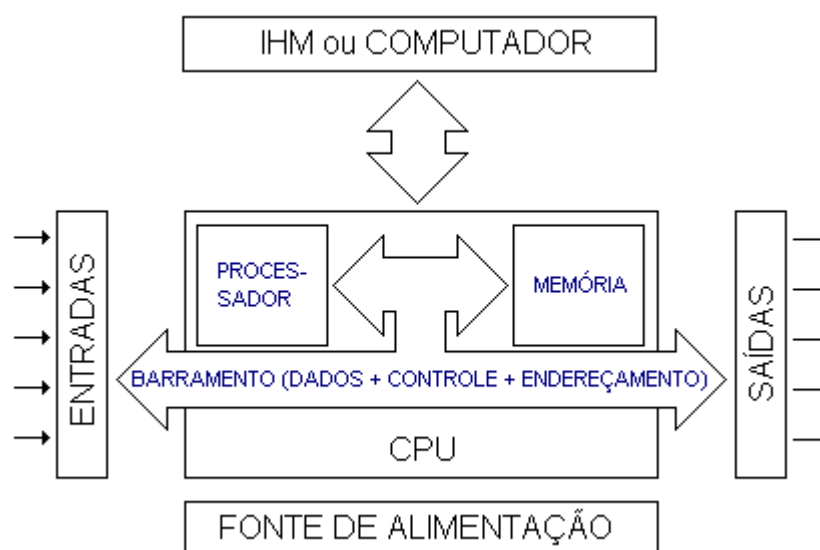


FIGURA 1 - Diagrama de blocos de um CLP

3.2 Resumo histórico

Segundo Georgine (2000, p.50), “Na década de 1960, o aumento da competitividade fez com que a indústria automotiva melhorasse o desempenho de suas linhas de produção, aumentando tanto qualidade como a produtividade.” A solução encontrada pela divisão *hydraulic* da General Motors foi substituir os sistemas de controle de relés por um sistema baseado no computador. Foi, então, que, em 1969, a Gould Modicon desenvolveu o primeiro CLP de acordo com os critérios estabelecidos pela GM.

Os novos equipamentos tinham preço competitivo em relação aos sistemas a relés, facilidade de programação e manutenção, bom funcionamento em ambiente industrial, dispositivos de entradas e saídas facilmente substituíveis e repetibilidade de operação de uso. Inicialmente, foram chamados de PC's - *Programmable Controllers*, mas com a popularidade dos computadores pessoais (PC's), convencionou-se chamar de PLC's - *Programmable Logic Controllers*, evitando conflitos de nomenclatura.

Na década de 1970, houveram os maiores aprimoramentos impulsionados pela evolução dos microprocessadores. Nesta época foram inseridas funções de temporização, contagem, operações aritméticas, controles analógicos, controle PID, comunicação, etc. Foi, também, sensivelmente elevada a capacidade de memória.

Apenas na década de 1980 surgiu a possibilidade de comunicação em rede e a redução acentuada nas dimensões dos CLP's. O uso em rede tornou o controle mais eficiente combinando o desempenho do controlador e a capacidade de controle distribuído de alta velocidade, além de melhorar a supervisão através de interfaces com computadores (CORRADI, 2005).

Atualmente, há no mercado diversos modelos de controladores programáveis, alguns bastante simples a exemplo dos primeiros modelos lançados, e outros muito sofisticados, desempenhando funções críticas e complexas, com entradas e saídas expansíveis, reduzido tempo de varredura, linguagens de programação de alto nível⁶, diagnósticos e detecção de falhas, operações matemáticas complexas, entre outras.

3.3 Classificação quanto ao gabinete

Os controladores estão disponíveis em diversas configurações e opções. Hackworth (2003, p.2-3) sugere dividi-los em três configurações físicas distintas: *frame open*, gabinete individual e modular. As configurações possuem características singulares relacionadas às funções disponíveis, ambiente indicado para operar, preço médio, etc.

3.3.1 *Frame open*

Os *frame open PLCs* (também chamados de *single board PLC's*) são modelos construídos em uma única placa de circuito impresso; convém observar que todos os componentes estão embutidos nesta placa (salvo a fonte de alimentação). Estas unidades, normalmente, não são expansíveis; algumas porém, permitem conexão via cabo com outras placas.

⁶ São linguagens distantes dos detalhes atrelados ao *hardware*. Um contra-exemplo seria a linguagem assembly que é muito próxima (intrínseca) ao *hardware*. Essa é considerada de baixo nível.

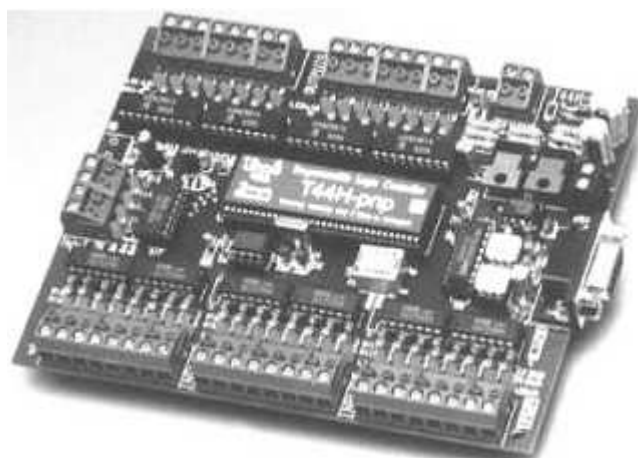


FIGURA 2 - Frame open PLC (Triangle Research Inc., Pte. Ltd.)

Fonte: HACKWORTH

Geralmente, os *frame open PLCs* são de baixo custo, com dimensões reduzidas e de baixo consumo. Por outro lado, muitas vezes, possuem poucas conexões de entrada e saída. São indicados para pequenas aplicações (HACKWORTH, 2003). A FIGURA 2 mostra uma fotografia de um CLP com esta configuração.

3.3.2 Gabinete Simples

São CLP's disponíveis em um único gabinete, como mostra a FIGURA 3, com conexões externas de entrada, saída e de alimentação. São escolhidos conforme a necessidade do projeto, quanto à memória disponível para aplicação, número de entradas e saídas e tensões suportadas. Normalmente, possuem uma porta de expansão que possibilita ligar unidades para aplicações específicas em alta velocidade, entradas e saídas analógicas, porta de comunicação, entre outros (HACKWORTH, 2003).



FIGURA 3 - CLP em gabinete individual (IDEC Corp.)

Fonte: HACKWORTH

3.3.3 Modular

As mais sofisticadas unidades CLP's são as modulares; um exemplo pode ser visto na FIGURA 4. Nestes equipamentos há uma CPU e um barramento onde podem ser conectados módulos conforme a necessidade do projeto. Quando instalado, mesmo com mudanças no projeto após a implantação, o equipamento pode ter sua estrutura física alterada (adicionando ou removendo módulos) e ser reprogramado.

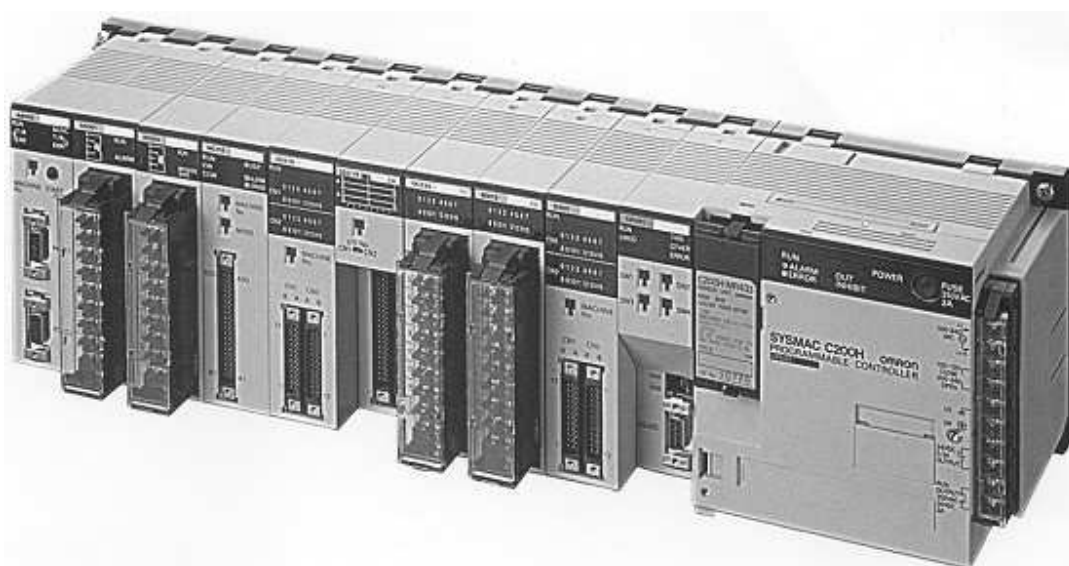


FIGURA 4 - CLP modular (Omron Eletronics)

Fonte: HACKWORTH

3.4 Classificação quanto ao tamanho

Bryan (1997) apresenta uma classificação baseada no tamanho do CLP.

São cinco possíveis: micro, pequenos, médios, grandes e CLP's muito grandes; de forma geral baseiam-se na quantidade de entradas e saídas. A FIGURA 5 mostra as categorias quanto ao tamanho e custo do CLP. O gráfico mostra áreas de intersecção entre categorias. Isso ocorre devido à presença de melhorias que adicionam opções em relação às características padrão da categoria. Estas opções podem elevar o CLP a uma categoria superior.

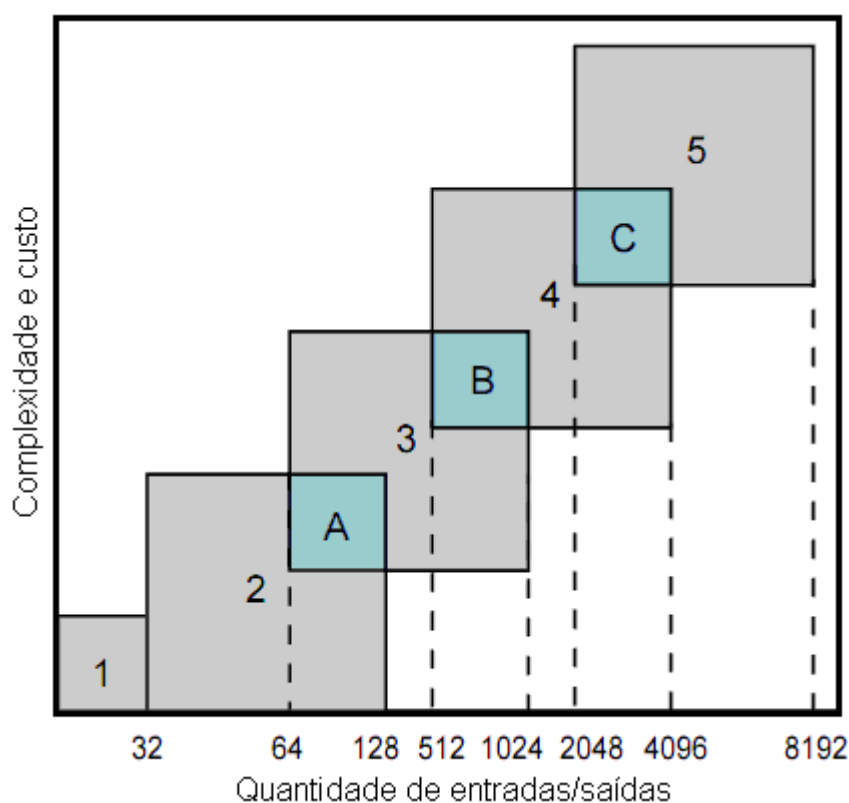


FIGURA 5 - Variações quanto ao tamanho

Fonte: BRYAN

Os micro CLP's são controladores com até 32 pontos de entrada e saída. Normalmente, possuem até 20 pontos com processador de 16 bits e memória de até 1 *kbyte*. Eles são, geralmente, programáveis através de uma Interface Homem-Máquina (IHM) e possuem contadores e temporizadores. São utilizados em pequenas aplicações.

A seguir, a categoria dos pequenos CLP's controlam entre 32 e 128 pontos de entradas e saídas digitais. Equipados com processador de 16 bits, memória de até 2 *kbytes*, suportam apenas programação *ladder* e booleana, possuem temporizadores, contadores e registradores de deslocamento. Podem ser

programados com IHM. Adicionalmente, podem estar presentes na área A (ver gráfico da FIGURA 5), subindo para categoria superior, desde que sejam capazes de executar controle analógico, operar matemática básica, possuir interfaces de rede, I/O's remotas, etc.

Os médios controlam entre 64 e 1024 entradas/saídas, substituem relés e controles analógicos, possuem processador de 16 ou 32 bits, memória de até 4kwords⁷ (expansível até 16k) e I/O's locais ou remotas. Podem ser programados com linguagem *ladder*, booleana, blocos de funções (linguagem de alto nível) e possuem porta de comunicação RS232. Além das características da categoria inferior, possuem capacidade de algumas poucas operações matemáticas e de tratamento de dados. A área B inclui dispositivos com maior quantidade de memória, PID, capacidade de operar sub-rotinas e instruções adicionais de matemática e tratamento de dados.

Os grandes CLP's trabalham com uma faixa entre 512 e 4096 pontos de I/O, são aplicados em tarefas de controle complexas que requerem manipulação de dados, aquisição e geração de relatórios. Através da integração com *software*, os dispositivos ainda podem executar tarefas mais complexas de matemática computacional. Somam ainda entradas e saídas analógicas, interrupções, módulos PID, operações de raiz quadrada, funções com matrizes, entre outras. A área C reserva CLP's com capacidades matemáticas e de tratamento de dados extras.

Em refinarias e siderúrgicas, aplicações que requerem grande número de entradas e saídas, controle sofisticado e enorme capacidade de memória, usam-se CLP's muito grandes que tratam de 2048 até 8192 pontos de I/O. Os processadores ou multiprocessadores são de 32 bits, memória de até 64 kwords (expansível a 1M), módulos remotos especiais, duas ou mais portas RS232, operações matemáticas com ponto flutuante, funções trigonométricas, além de LIFO⁸ e FIFO⁹ (BRYAN, 1997).

7 Uma *word* tem tamanho igual a quantidade de bits que a CPU pode processar em um ciclo (INDUSTRIAL, 2002). Normalmente, neste caso, trata-se de 16 ou 32 bits.

8 LIFO: último dado a entrar é o primeiro a sair.

9 FIFO: primeiro dado a entrar é o primeiro a sair.

3.5 Norma IEC 61131

A norma IEC 61131 é o resultado do esforço da Comissão Internacional de Eletroeletrônicos em definir padrões de arquitetura e métodos de programação para CLP's. Aplicar o padrão não tem sido tarefa fácil, pois, em alguns modelos, os fabricantes têm dificuldade na adaptação. Por outro lado, o padrão já causou grande impacto que repercutirá na maneira em que os CLP's serão programados no futuro (BRYAN, 1997). A norma que era codificada como IEC 1131, teve, mais recentemente, associado o número 6.

Segundo Silveira e Santos (1998, p.218), a norma se destaca na “tentativa de uniformização das linguagens de programação existentes e disponíveis, por meio dos inúmeros fabricantes atuantes no mercado mundial”. É o que delibera o terceiro item da norma. A TABELA 1 mostra as partes que compõem a norma.

TABELA 1 - Partes da norma IEC 61131

Parte	Título	Conteúdo
1	<i>General Information</i>	Definição da terminologia e conceitos.
2	<i>Equipment requirements and tests</i>	Teste de verificação e fabricação eletrônica e mecânica.
3	<i>Programmable Languages</i>	Estrutura do software do CLP, linguagens e execução de programas.
4	<i>User guidelines</i>	Orientações para seleção, instalação e manutenção de CLP's.
5	<i>Messaging service specification</i>	Funcionalidades para comunicação com outros dispositivos.
6		Reservada
7	<i>Fuzzy Control Programming</i>	Funcionalidades de software, incluindo blocos funcionais padrões para tratamento de lógica nebulosa dentro de CLP's.
8	<i>Guidelines for the implementation of languages for programmable controllers</i>	Orientações para implementação das linguagens IEC 1131-3.

Fonte: IEC61131

3.6 Seqüência de operação

“Todos os CLP's possuem quatro estágios básicos de operação que são repetidos muitas vezes por segundo” (JACK, 2004, p.8.3, tradução nossa). No momento em que o dispositivo é energizado, ao ligar, são realizadas rotinas de inicialização que, por exemplo, zeram as variáveis de memória e desligam as saídas. A FIGURA 6 mostra esses quatro estágios executados ciclicamente, além do estágio de inicialização.

Ao iniciar o ciclo, também chamado de *scan*, primeiramente é realizado uma verificação por falhas de *hardware* e *software*, em seguida é zerado o *watchdog*¹⁰ e atualizado o *display*¹¹. Se ocorrer uma falha recorrente nas funções básicas o sistema, por segurança, simplesmente não progride.

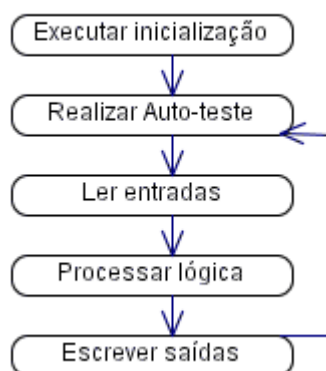


FIGURA 6 - Seqüência de operação

Em seguida, o sistema faz a leitura das entradas. Os valores lidos nos pinos físicos são copiados para variáveis de memória que simbolizam as entradas. Este processo é importante para agilizar o processamento do dado e para garantir que o valor da variável não se altere até o término do ciclo. Isso evita divergências no caso desse valor ser utilizado em múltiplas chamadas.

Utilizando as variáveis de memória, o programa aplicação é processado, os valores de saída são então copiados para variáveis de memória que representam as saídas. Esta lógica baseia-se na tabela de valores em memória e os comandos

¹⁰ *Watchdog* é uma função normalmente já incorporada na CPU que reinicia o sistema se esse não estiver respondendo por muito tempo.

¹¹ O *display* é o visor, um opcional que pode estar mostrando um valor de contagem, relógio, etc.

são executados passo-a-passo na ordem em que foram programados.

Por fim, os valores gravados nas variáveis temporárias são copiados para os pinos de saída. Ao atualizar as saídas físicas, o CLP está atuando sobre os equipamentos conectados a ele. As alterações nas saídas, muitas vezes, provocam mudanças nas entradas que serão processadas no próximo ciclo.

3.7 Linguagens de programação

Desde a sua criação, tanto o CLP quanto as linguagens de programação têm evoluído. A linguagem permite ao usuário introduzir um programa a ser executado, de acordo com uma sintaxe pré-estabelecida. As diferentes linguagens surgiram de acordo com as distintas aplicações do controlador.

O programa escrito utilizando instruções textuais ou gráficas é chamado programa-fonte. Todo código fonte precisa ser traduzido para a linguagem binária utilizada pelo CLP. O *firmware* do CLP, portanto, interpreta uma única linguagem que é complexa por ser de representação numérica. O arquivo traduzido nessa linguagem é chamado programa-objeto.

“O padrão IEC 61131-3 define duas linguagens gráficas e duas baseadas em texto para programar CLP's” (BRYAN, 1997, p. 375, tradução nossa).

Linguagens gráficas:

- Diagrama *Ladder*
- Diagrama de Blocos Funcionais

Linguagens baseadas em texto:

- Lista de instruções
- Texto estruturado

Somando-se a estas linguagens o padrão IEC 61131-3 inclui um método de programação orientado a objetos chamado Diagrama de Funções Seqüenciais (SFC). Em alguns casos, o SFC é considerado uma linguagem de programação mas, na realidade, é uma estrutura organizacional que se utiliza das quatro

linguagens verdadeiras (BRYAN, 1997).

Muitos dos controladores modernos suportam programas escritos em diferentes linguagens, desde que divididos em blocos. Através da programação em blocos, todas as linguagens IEC 61131-3 podem ser utilizadas dentro de uma mesma aplicação desde que o *software* compilador suporte esta característica.

Para Silveira e Santos (1998), nem todas as instruções de programação podem ser representadas em qualquer tipo de linguagem. Apesar do diagrama *ladder* ser bastante claro aos técnicos da área elétrica, bem como o diagrama de blocos funcionais aos da área eletrônica, na programação de um CLP, a forma mais condizente com o modo como os processadores operam é a lista de instruções.

A lista de instruções surgiu para reduzir a dificuldade que se tem ao manipular-se representações numéricas (códigos). Nada mais é que a associação de símbolos (nomes e rótulos) às instruções codificadas. Um programa escrito com auxílio de símbolos, ou menmônicos, precisa ser traduzido para a linguagem codificada para que possa ser executado. Segundo Weber (2004, p. 36), “essa tradução recebe o nome de montagem e, quando automatizada, o programa que realiza recebe o nome de montador”.

Weber (2004, p.37) esclarece: “um montador realiza praticamente apenas uma tradução 'um para um' das instruções da linguagem simbólica para instruções de máquina”. No caso do compilador Weber diferencia dizendo que esse “gera rotinas em linguagem de máquina para cada instrução da linguagem de alto nível e depois otimiza código e alocação de variáveis”.

3.7.1 Diagrama Ladder (LD)

O diagrama *Ladder* é o mais antigo e, provavelmente, o método mais comum de programação descrito na norma IEC 1131-3 (PARR, 2003). Trata-se de uma representação gráfica que descreve a lógica através de símbolos de contatos e bobinas de relés. Os símbolos e a disposição são heranças da antiga forma de descrever quadros de relés.

O diagrama é limitado por linhas verticais no lado esquerdo e direito

representando trilhas de tensão. Cada item possui um valor booleano associado (verdadeiro/ligado ou falso/desligado). A sequência deve ser lida da esquerda para a direita e de cima para baixo. A FIGURA 7 mostra um pequeno programa em *Ladder*, neste exemplo se o “Botão1” ou “Botão2” forem acionados e o “Sensor” também estiver ativado o “Motor” é ligado. Se o motor estiver ligado um sinal sonoro será emitido.

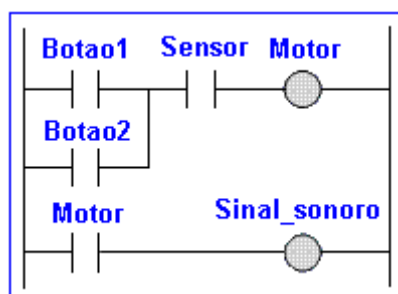


FIGURA 7 - Exemplo de um programa *Ladder*

3.7.2 Diagrama de Blocos Funcionais (FBD)

O Diagrama de Blocos Funcionais é uma forma gráfica de programar o CLP nos mesmos moldes em que são descritos circuitos elétricos. O conceito principal é o fluxo de dados onde esses vão das entradas para as saídas através dos blocos de funções (JACK, 2004). As funções são definidas através de blocos característicos, a IEC 61131 prevê blocos que realizam funções básicas e avançadas. “As funções básicas usadas em um programa FBD são equivalentes ao conjunto básico usado em um programa de Texto Estruturado” (JACK, 2004, p. 21.2, tradução nossa). Alguns fabricantes incluem blocos próprios definidos para dispositivos específicos.

Além dos blocos padrão e específicos de alguns fabricantes, o usuário ainda pode criar seus próprios blocos de acordo com suas necessidades. A criação de blocos personalizados chama-se encapsulamento. Para criar blocos o usuário pode utilizar outros blocos existentes, lista de instruções, programação em texto estruturado com ou sem instruções *ladder*. Ainda é possível incluí-los em uma biblioteca, onde podem ser acessados por outros projetos, a exemplo do uso dos blocos padrão (BRYAN, 1997).

A FIGURA 8 mostra um circuito de controle utilizando temporizador,

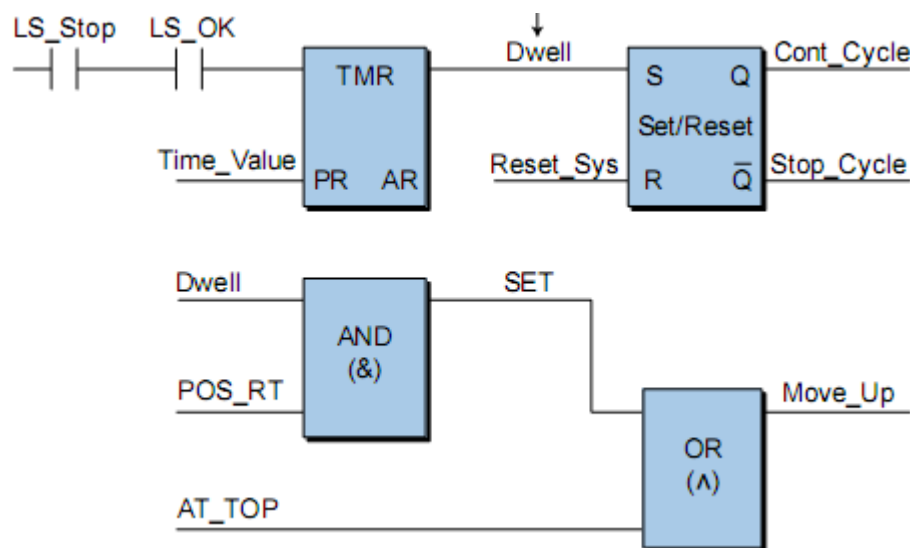


FIGURA 8 - Exemplo de uso do FBD

Fonte: BRYAN

lógicas “E” e “OU” e um bloco do tipo SET/RESET. Neste exemplo, o temporizador seta uma saída e, também, pode comandar um movimento dependendo da condição de outras variáveis.

No exemplo, o temporizador é disparado dependendo do resultado da lógica “E” entre “LS_Stop” e “LS_OK”, neste caso utilizando a mesma simbologia da linguagem *ladder*. Ao estourar o temporizador, são ativados o bloco SET/RESET e a variável “Dwell”, essa é utilizada no circuito como entrada de um bloco lógico “E” juntamente com uma outra variável “POS_RT”. O resultado dessa lógica e a variável “AT_TOP” são as entradas de um bloco lógico “OU”, que, finalmente, atua na variável “Move_Up”. Não há saídas do tipo relé, portanto seriam lógicas para serem tratadas posteriormente em um programa aplicação.

3.7.3 Lista de Instruções (IL)

Também chamada de *Statement List* (Lista de Sentenças), trata-se de uma linguagem de baixo nível similar a linguagem *assembly* utilizada em microprocessadores. O código é composto por um conjunto de mnemônicos booleanos que são executados sequencialmente. Muito utilizada em pequenas aplicações, como também em aplicações que requerem máxima performance do CLP ou em rotinas específicas (BRYAN, 1997).

Para exemplificar a FIGURA 9 mostra um exemplo de código que faz lógica “E” (booleana) entre uma entrada e o resultado de uma lógica “OU” entre duas outras entradas, tendo uma delas seu valor negado. O resultado é gravado na variável “B0”.

LD	B1	(*resultado atual := B1*)
AND(	B2	(*B1 é guardado na pilha*)
ORN	B3	(*resultado atual := B2 OR NOT B3*)
)		(*resultado atual := B1 AND (B2 OR NOT B3*)
ST	B0	(*B0 := resultado atual*)

FIGURA 9 - Exemplo de uma lista de instruções

A IL aceita instruções do tipo “AND(“ e “OR(“, ou seja, instruções que trabalham uma sentença dentro de uma operação, ou outra sentença, limitada por parênteses. Para entender é necessário introduzir o conceito *stack*, que é uma memória que guarda valores na forma de uma pilha “empurrando” e “puxando” os dados ordenadamente. Assim, no caso exemplificado, B1 é “empurrado” para o *stack* quando o programa está na linha contendo “AND(“, segue-se o processo da lógica “OU” entre B2 e B3 (esse negado), ao encontrar o fechamento do parênteses o interpretador “puxa” o valor do *stack* para realizar a lógica “E”.

Todas as outras linguagens de programação podem ser convertidas para IL. Portanto há instruções IL para descrever todas as operações disponíveis no CLP, sejam elas básicas ou avançadas.

3.7.4 Texto Estruturado (ST)

Texto Estruturado é uma linguagem textual de alto nível, similar ao PASCAL ou BASIC, desenvolvida para programação de processos em automação. Utilizada principalmente na representação de procedimento complexos que não podem ser facilmente expressos em linguagens gráficas. Tarefas complexas podem ser divididas em tarefas menores e mais simples.

Ela se aproxima da linguagem SFC por poder descrever ações como estágios. Assim como LD, FBD e IL, utiliza definições de variáveis para identificar

entradas, saídas e outros registros internos. A linguagem suporta iterações como WHILE...DO e REPEAT...UNTIL, além de condições do tipo IF...THEN...ELSE.

O Texto Estruturado se adapta muito bem em aplicações que envolvem manipulação de dados, classificação de informações e funções matemáticas envolvendo ponto flutuante (BRYAN, 1997). Um exemplo de pequeno código escrito em linguagem estruturada pode ser visto na FIGURA 10. Neste exemplo, se a variável “mostra_contagem” for verdadeira, uma mensagem será escrita mostrando o valor atual de uma determinada contagem.

```
IF mostra_contagem THEN
    Message “Contagem atual:” := valor_contagem
END_IF
```

FIGURA 10 - Exemplo de código ST

3.7.5 Diagrama de Funções Sequenciais (SFC)

O Diagrama de Funções Sequenciais originou-se do padrão francês chamado Grafcet (*Graphe Fonctionnel de Commande Étape Transition*) ou IEC 848 (BRYAN, 1997). “Uma linguagem de programação de CLP que usa orientação a objetos, estrutura semelhante a um fluxograma, juntamente com passos, transições e ações para definir um programa de controle” (INDUSTRIAL, 2002, tradução nossa). Jack (2004) completa dizendo que é uma técnica gráfica capaz de descrever programas de controle concorrentes. No caso desses programas, requer-se um controlador que seja capaz de processar tarefas em paralelo, o que significa que mais de um estado pode estar ativo ao mesmo tempo.

Os gráficos utilizam três componentes básicos: passos, transições e ações.

- Passo¹²: é um estágio no controle de um processo. O primeiro passo é simbolizado de forma especial. Um passo pode ou não ter uma ação associada.
- Transição: após um passo/ação há uma transição antes do próximo passo.

¹² O passo aqui descrito não se refere ao termo “passo” do CLP, esse diz respeito a uma instrução e seu argumento.

Uma transição pode também conter uma sentença condicional.

- Ação: é um conjunto de instruções de controle que são executadas durante um passo.

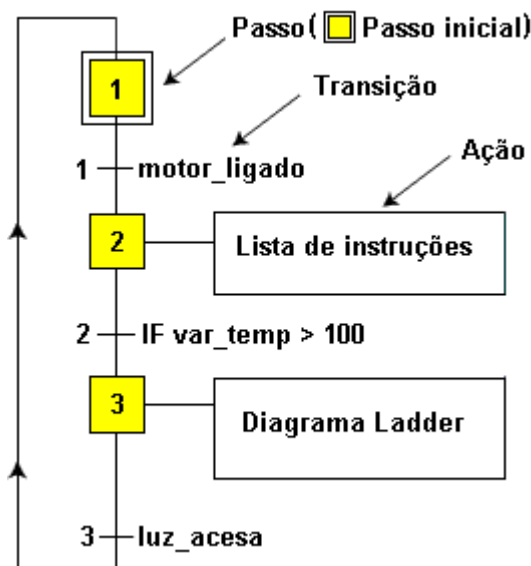


FIGURA 11 - Exemplo de diagrama de funções seqüenciais

Na FIGURA 11 é ilustrado um SFC misto que utiliza também uma IL e LD. A possibilidade de agregar outras linguagens em um diagrama SFC é a principal diferença entre esse e o Grafcet. O SFC pode ser pensado para implementação do programa completo, adicionados a esse, outros pequenos blocos de programa seriam escritos em quaisquer outras linguagens.

Nesse exemplo, após o passo inicial, a variável “motor_ligado” é ativada durante a transição ao segundo passo. Nesse passo é executada uma Lista de Instruções alterando o valor da variável “var_temp” que, na transição, é comparada para saber se supera a constante igual a “100”. Se o valor for superior a “100”, no terceiro passo é executado um Diagrama *Ladder*. Finalmente, na transição seguinte a variável “luz_acesa” é ativada.

3.8 Protocolos de comunicação

Um protocolo de comunicação é um “conjunto de regras, procedimento e leis que governam a troca de informação entre dois ou mais processos, incluindo o

formato e ações a serem executadas quando do envio e do recebimento desses dados” (SILVEIRA; SANTOS, 1998, p.183). O protocolo, portanto, deve ser conhecido por todos os elementos comunicantes da rede. No protocolo estão definidas funções específicas para realização de tarefas como estabelecimento de sincronismo, envio de estado, atuação sob periféricos e tratamento de erros.

Segundo Silveira e Santos (1998), todos os protocolos possuem as seguintes funções:

- Endereçamento: parte da mensagem que informa o destino dessa na rede.
- Estabelecimento de conexão: também chamado de *handshaking*¹³, é procedimento inicial para estabelecimento de uma comunicação e execução de uma tarefa.
- Confirmação de recebimento: mensagem em que o destinatário confirma o recebimento bem sucedido de uma mensagem anterior.
- Pedido de retransmissão: no caso da mensagem recebida conter erros, deve haver algum recurso para solicitação de reenvio.
- Conversão de código: para o caso da mensagem que trafega pela rede ter uma codificação diferente da codificação utilizada pelo destinatário.
- Numeração e seqüência: recurso que permite dividir uma mensagem grande em partes numeradas para que o destinatário possa recompô-la.
- Controle de fluxo: “define o tamanho máximo da mensagem para ser transportada na rede de comunicação. É definido em função do tamanho da mensagem, o número de blocos em que ela será fatiada” (SILVEIRA; SANTOS, 1998, p.184).

3.8.1 O protocolo MODBUS

“MODBUS é um protocolo de mensagens de camada de aplicação, posicionado no sétimo nível do modelo OSI (*Open Systems Interconnection*), que permite comunicação cliente/servidor entre dispositivos conectados em diferentes

¹³ Troca de sinais predeterminados entre máquinas coligadas para confirmar a coligação recíproca (GARDINI; LIMA, 2003)

tipos de barramentos e redes” (MODBUS, 2004, p.2, tradução nossa). Foi criado pela empresa Gould Modicon, hoje Schneider Eletric (REYNDERS, 2005).

Desenvolvido na década de 1970, é um sistema de comunicação tipo “mestre-escravo” onde há um nó (nó mestre) que lança comandos explícitos para um nó “escravo” e processa as respostas dos escravos. Os nós escravos normalmente não transmitem dados sem requisição do nó mestre e não podem se comunicar com outros escravos (MODBUS, 2002). O mestre também pode ser visto como cliente e os escravos como servidores num formato de comunicação requisição/resposta.

No nível físico, sistemas MODBUS sobre linha serial podem usar diferentes interfaces físicas, por exemplo RS232 ou RS485. No padrão RS485, interfaces a dois fios são mais comuns e interfaces a quatro fios também podem ser implementadas (MODBUS, 2004). O padrão RS232 é usado apenas em distâncias curtas de aplicações ponto a ponto. Na comunicação serial, a cada *byte* transmitido há 1 bit de inicialização, 8 de dados, 1 de paridade e 1 de parada (MODBUS, 2002).

Conforme a FIGURA 12, a mensagem é formada por uma unidade de dados fundamental contendo o código da função e os dados cujo tamanho é definido pela própria função. Esta parte da mensagem pode conter até 253 *bytes* (MODBUS, 2004). A mensagem de aplicação completa pode ter até 256 *bytes* na comunicação serial, portanto, contém adicionalmente o endereço do escravo e *checksum*¹⁴ de 2 *bytes*.

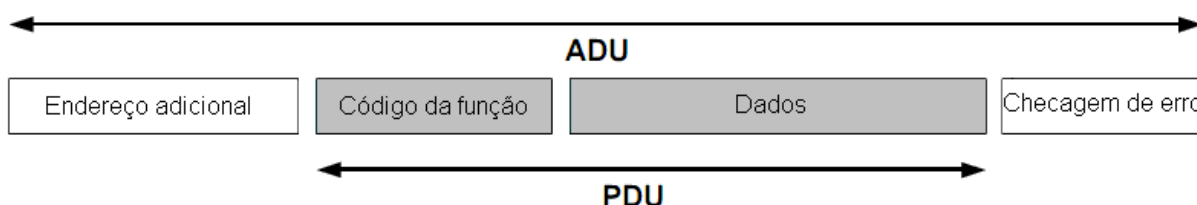


FIGURA 12 - Composição da mensagem MODBUS

Fonte: MODBUS

A função é informada em um *byte* através do campo código da função, cujo valor pode variar de 1 a 255. As funções públicas definidas para o protocolo são apresentadas no ANEXO A. Os valores entre 128 e 255 são reservados para

¹⁴ Um algoritmo matemático que adiciona um resultado binário de todos os caracteres em um bloco de dados e coloca o cálculo em um bloco de checagem (INDUSTRIAL, 2002).

exceções. Para endereçamento, o *byte* disponível tem reservados os valores de 248 a 255, além do valor 0 ser de *broadcast*. Toda a faixa entre 1 e 247 está disponível para endereçamento dos escravos.

Na troca de mensagens, o mestre envia solicitações ao escravo que são reconhecidas pelo código da função. Este código ocupa 1 *byte* da mensagem e determina a quantidade de dados adicionais. O escravo responde retornando o mesmo código de função. Em caso de erro, o escravo substitui o código da função por um código que indica a falha; nesta mensagem é incluído um *byte* que detalha o tipo de erro ocorrido (SOUZA, 2004).

3.8.2 O protocolo MODBUS e o modelo OSI

O modelo OSI (*Open Systems Interconnection*) é uma descrição da comunicação em rede organizada em sete camadas desenvolvido para interconexão entre sistemas abertos¹⁵ (INDUSTRIAL, 2002). O protocolo MODBUS se adequa bem a essa estrutura, porém nem todas as camadas são implementadas, como mostra a TABELA 2.

TABELA 2 - Estrutura de camadas do modelo OSI e camadas do MODBUS

Camada	Modelo ISO/OSI	
7	Aplicação	Protocolo de aplicação MODBUS
6	Apresentação	Vazia
5	Sessão	Vazia
4	Transporte	Vazia
3	Rede	Vazia
2	Enlace	Protocolo em linha serial MODBUS
1	Física	EIA/TIA-485 (ou EIA/TIA-232)

Fonte: MODBUS

- Camada física: neste caso o padrão RS485. Há um par de fios em que o mestre envia dados que todos os escravos recebem. Um outro par torna o mestre capaz de receber pacotes dos escravos.
- Camada de enlace: determinada pela velocidade, configuração de paridade, bit de parada e ordenação da UART.
- Camada de aplicação: espaço cujo uso depende dos equipamentos ligados à rede. Cada equipamento deverá utilizar algumas das funções já definidas de

¹⁵ Não proprietários, geralmente utilizado por várias empresas

leitura e escrita de registros, banco de dados, sinalização, etc.

No caso específico do CLP a camada de aplicação é utilizada especialmente no acesso a registros. O mestre neste caso pode ser visto como o *software* editor que está rodando no computador. Apenas um escravo é conectado, nesse caso, à placa do CLP. O *software* acessa os registros da memória EEPROM da placa e permite gravação de novos dados sob comando do usuário.

As demais camadas não são implementadas pelo protocolo MODBUS. A camada de rede seria responsável por estabelecer, manter e terminar conexões quanto à tecnologia de transmissão de dados empregada. A camada de transporte seria responsável pela transferência de dados entre dois pontos de forma transparente e confiável. A camada de sessão seria responsável por estabelecer, manter e gerenciar conexões entre aplicações. Por fim, a camada de apresentação proveria independência aos processos tratando a sintaxe das mensagens.

4 PROJETO DO SOFTWARE E FIRMWARE DO CLP

O projeto especifica um Controlador Lógico Programável implementado utilizando um microcontrolador da família 8051 com 16 entradas digitais e 8 relés de saída, capaz de realizar lógicas booleanas, contagens e temporizações. Através dos requisitos definidos para o projeto e das definições de *hardware* é determinado como ficam dispostas as variáveis do programa do *firmware* e o formato do programa aplicação, que foi definido pelo usuário e enviado através do *software* de programação.

O *firmware*, gravado na memória flash do microcontrolador, é responsável pela varredura das entradas, processamento dos dados conforme interpretação da seqüência do programa aplicação, escrita das saídas e comunicação com o microcomputador. O programa aplicação é gravado na memória não volátil EEPROM. A memória RAM é utilizada para guardar valores temporários das entradas e saídas, valores de contagens, de temporizações, etc.

4.1 Requisitos do projeto

Necessita-se desenvolver um CLP que seja implementado para rodar em um microcontrolador da família 8051, em linguagem C (para facilitar futura manutenção) e deve utilizar uma placa já disponível com 16 entradas e 8 relés de saída. Ainda é requerido que possibilite programação via computador e que desempenhe as principais funções de um controlador lógico programável comum.

Vislumbra-se a utilização do CLP em projetos de segurança que requerem customização, para realização de inter-travamentos de portas e em linhas de montagem de empresas. Os principais usuários potenciais do produto são bancos, presídios e grandes empresas, além de condomínios e residências.

O diagrama de casos de uso do projeto, conforme FIGURA 13, prevê três atores: o “Projetista” responsável pelo programa aplicação, o “Software” e, finalmente, o “CLP”. O Projetista deve escrever o programa aplicação através do *software*. O Software é capaz de traduzir os comandos e enviar ao CLP, que, por sua vez, deve ser capaz de interpretar os comandos e realizar contagens,

temporizações, lógicas booleanas, comparações entre números e comunicação com o PC.

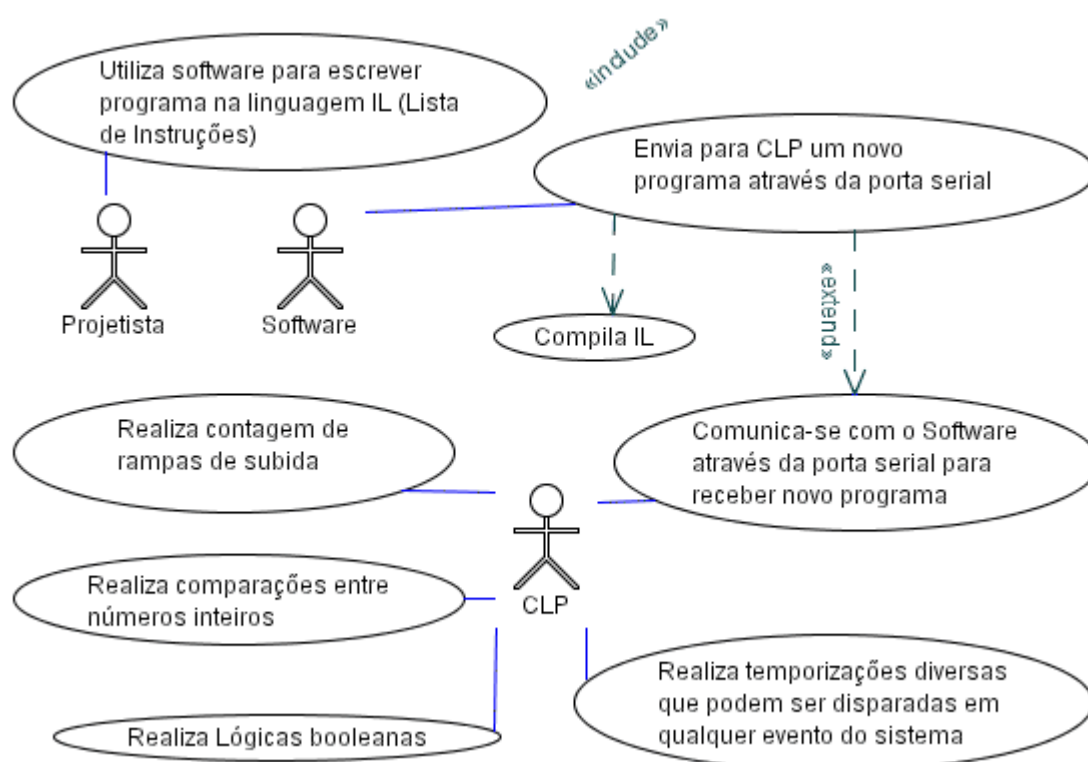


FIGURA 13 - Diagrama de caso de uso

4.2 Configuração do hardware

A placa foi definida previamente reaproveitando-se uma placa já disponível. A fonte de alimentação que fornece energia à placa foi também definida previamente, conforme requisitos ou demanda de corrente da placa. Demais equipamentos que podem ser comandados pelo CLP devem ser alimentados por fontes externas.

A especificação do microcontrolador foi obtida percebendo-se a necessidade prática em termos de performance e, especialmente, o uso de memória RAM. A restrição inicial é que deveria ser um microcontrolador da família 8051 ou compatível, já que a placa disponível foi projetada para essa família.

A memória EEPROM escolhida depende do espaço que as instruções do CLP requisitam. Outro fator preponderante para definição da capacidade de armazenamento é o tamanho máximo que poderá ocupar uma aplicação. O próprio

desempenho do CLP influencia no tamanho máximo da memória de aplicação pois não pode haver um número de instruções tão elevado que comprometa o tempo de *scan* e temporizadores.

4.2.1 A placa

A placa utilizada no projeto (FIGURA 14) foi desenvolvida para atender a situações específicas, programada diretamente no microcontrolador 8051. Não se esperava exatamente desenvolver um dispositivo programável, mas a PCI veio a se tornar uma boa plataforma para portá-lo. A placa deve ser alimentada com 12V e possui regulador de tensão de 5V. O esquemático da placa é mostrado no APÊNDICE B.

Outros itens são: soquete para 8051 (40 pinos) e circuitos adicionais, soquete para memória serial EEPROM, soquete e bateria para RTC (*Real Time Clock* – Relógio de Tempo Real), comunicação RS-485, 4 saídas para LED's ou Buzzer's e 1 conexão bidirecional (pode ser utilizada tanto para entrada como para saída de dados).

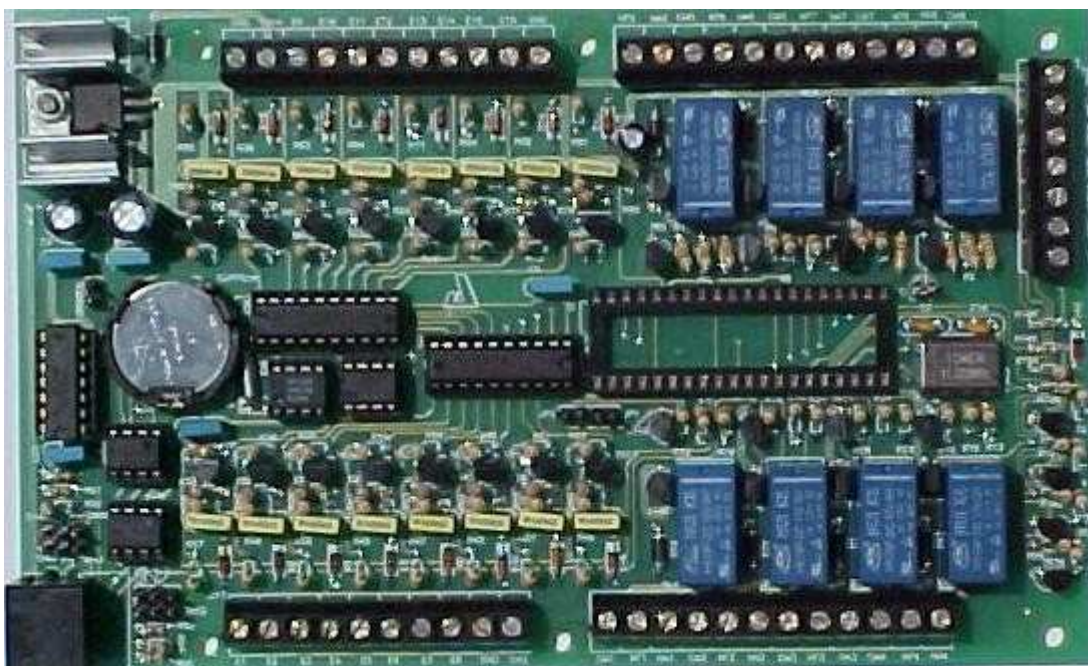


FIGURA 14 - Fotografia da placa

Os 8 relés de saída e 16 entradas digitais são acessíveis ao usuário

através de conectores. Para as entradas, sensores e chaves são ligadas entre os conectores E1 a E16 e conexão GND. Cada um dos 8 relés têm disponíveis para conexão os pinos NA, NF¹⁶ e COMUM.

Este projeto utiliza o microcontrolador 89C51ED2 que possui 64 *kbytes* de memória flash e 2 *kbytes* de memória RAM. O referido componente pode ser 2 vezes mais veloz que o original 8051. Também é utilizada uma memória EEPROM serial do tipo 24LC64 com capacidade de armazenamento de 8 *kbytes* de informações. Outros detalhes estão descritos no ANEXO C.

4.2.2 Configuração da memória volátil

Há um total de 2048 *bytes* de memória RAM, onde 128 são de *data*, 128 *idata* (somando 256 de memória “interna”) e 1792 *xdata* (acessados como memória “externa”). Apesar de parte da memória ser acessada como memória externa, toda memória disponível é inerente ao componente 89C51ED2.

Grande parte do espaço disponível já está sendo utilizado pelos componentes do CLP. Na TABELA 3 são apresentados os recursos do CLP e a quantidade de espaços de memória que ocupam. Os itens entradas, saídas e *flags* são os mais simples e envolvem basicamente um espelho de memória e *bytes* adicionais declarados na forma de variáveis utilizadas no processamento da leitura e escrita. Os recursos “temporizadores” e “contadores” envolvem muitas funções e variáveis no processamento, por esta razão, além de suas saídas, outros sub-itens e o processamento requerem uso da memória volátil.

16 NA e NF, Normalmente Aberto e Normalmente Fechado, respectivamente.

TABELA 3 - Ocupação da memória RAM

Item	Sub-item	Quantidade	Espaço unitário (bits)	Espaço Total (bytes)	Sub-total
Entradas	Espelho	16	1	2	
	Processamento	1	41	5	7,1
Espelho das Saídas	Espelho	8	1	1	
	Processamento	1	66	8	9,3
Temporizadores	Valor atual	64	16	128	
	Valor de preset	64	16	128	
	Flag de Overflow	64	1	8	
	Flag de Bloqueio	64	1	8	
	Flag de Saída	64	1	8	
Contadores	Processamento	1	236	30	309,5
	Valor de contagem	64	16	128	
	Flag de saída	64	1	8	
	Flag valor UP	64	1	8	
	Flag valor DOWN	64	1	8	
	Processamento	1	189	24	175,6
Flags	Espelho	256	1	32	
	Processamento	1	74	9	41,3
Total					542,8

A ocupação é contada em *bytes*, devido fato de que quase todos os recursos precisam ser contados em bits, a tabela calcula a quantidade do recurso multiplicando pelo espaço unitário em bits. O resultado é o espaço total ocupado em bits que por fim é dividido para expressar em *bytes*. Para cada recurso o item “Processamento” representa a quantidade de memória utilizada para tratamento desse recurso. Todos estes dados foram obtidos após a compilação do programa final.

O microcontrolador 89C51ED2, portanto tem ocupado 542,8 bytes de sua memória RAM, o que representa 26% de ocupação dos 2048 *bytes* da RAM. Outras implementações, como o laço principal, tratamento da memória EEPROM e protocolo de comunicação, também ocupam espaço da RAM. Neste caso, a tabela apresenta os dados apenas relativos aos recursos para facilitar em estimativas de expansão.

4.2.3 Configuração da memória de aplicação

O programa aplicação fica armazenado na memória EEPROM serial que, por ser uma memória não volátil, permitirá que o CLP guarde os comandos mesmo com ausência de energia na placa. Ao ser inicializado, o CLP lê a memória retentiva

e interpreta os comandos. Dos 8192 *bytes* disponíveis nessa memória, 64 estão reservados para configurações. Os 8128 restantes são para gravação do programa aplicação, o que permite armazenamento de aproximadamente 2700 instruções.

Conforme a TABELA 4, os espaços de memória 0x0000 e 0x0001 são utilizados para gravação da versão da tabela de alocação de dados. Isto permitirá que no futuro seja alterada mais facilmente a forma de dispor informações na memória. Os 16 *bytes* entre 0x0002 e 0x0011 guardam o valor padrão (zero ou um) de cada saída. As saídas recebem esses valores no instante em que o firmware está sendo inicializado. Os 16 *bytes* seguintes, entre 0x0012 e 0x0021, estão reservados para, no futuro, configurarem o dispositivo para gravar eventos quando uma *flag* mudar de estado.

TABELA 4 - Configuração da memória serial EEPROM

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	Versao	Versao	PS*	PS*	PS*	PS*	PS*	PS*	PS*	PS*	PS*	PS*	PS*	PS*	PS*	PS*
0010	PS*	PS*	EF*	EF*	EF*	EF*	EF*	EF*	EF*	EF*	EF*	EF*	EF*	EF*	EF*	EF*
0020	EF*	EF*														
0030																
0040																
0050																
...																
1FE0																
1FF0																

Legenda (0x2000) equivale a uma memória de 8 kbytes

64 bytes reservados para configurações do firmware

Programa Aplicação

PS* Valor padrão (bit a bit) para saídas (inicialização e em caso de acionamento do botão de pânico)

EF* Seta geração de eventos (bit a bit) por flag (ao mudar o estado – requer RTC)

A TABELA 5 mostra um exemplo de uso da memória EEPROM gravando um pequeno programa traduzido para a linguagem binária interpretada pelo CLP. Os espaços 0x0000 e 0x0001 mostram que a alocação está disposta conforme versão 0.1, que é justamente a que prevê a configuração descrita anteriormente. O pequeno programa aplicação deste exemplo está gravado entre os *bytes* 0x0040 e 0x0050, é composto pelo total de 17 *bytes*. Os valores são expressos em base hexadecimal.

TABELA 5 - Exemplo de ocupação da memória EEPROM

	MEMÓRIA EEPROM															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	0	1	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0010	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0020	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0030	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0040	2	2	0D	0B	5	10	1E	12	3	1	1	3	1	0A	0	7
0050	1F	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
...	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
1FE0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
1FF0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

4.3 Conjunto de instruções

O conjunto de instruções baseia-se na linguagem STEP5 utilizada, sobretudo, por equipamentos da Siemens. O conjunto de instruções da linguagem STEP5 está descrito no ANEXO B. As instruções implementadas neste projeto permitem a execução de lógicas booleanas AND, OR e XOR, configuração e execução de temporizadores e contadores, instruções de comparação de números inteiros, carga e transferência de números inteiros e de controle da aplicação. O conjunto dessas instruções está sintetizado no APÊNDICE A.

Cada instrução é vista pelo *software* montador e *firmware* interpretador como formada por um *byte* de operação obrigatório, outro *byte* representando um operador e por último um *byte* de argumento. Algumas instruções podem não requerer operador e argumento, conforme simbolizado pela FIGURA 15.

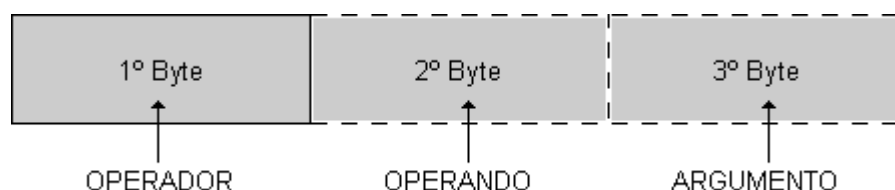


FIGURA 15 - Formato da instrução

As operações serão vistas detalhadamente a seguir, os argumentos variam de acordo com o operador. Os seguintes operadores estão definidos:

- Q: relé de saída.
- F: flag de uso geral.
- I: entrada do CLP.
- T: flag de saída (resultado) do temporizador.

- C: flag de saída (resultado) do contador.
- K: valor constante.

As conexões de entrada e saída do CLP são divididas em grupos de 8 pinos, cada grupo aqui será chamado de porta. Os argumentos dos operadores Q, F e I podem variar de 0.0 a 31.7 totalizando 256 possibilidades. O algarismo que antecede o ponto representa o *byte* ou uma porta, o algarismo posterior ao ponto representa o bit específico do *byte*, que é o pino de conexão no caso de entradas e saídas físicas.

Alguns exemplos seriam: a sétima Flag seria representada pelos operador e argumento “F 0.6”, pois o índice começa a contar de algarismo “zero”, portanto o número “6” representa o sétimo elemento. A vigésima entrada 20 seria representada pelo operador e argumento “I 2.3”, devido o algarismo anterior ao ponto ser “2” que é multiplicado por 8 (número de bits do byte) e somado a “4”, já que começa a contar do algarismo “zero”, a conta fica portanto, $2 \times 8 + 4 = 20$.

Originalmente o projeto disponibiliza 16 entradas que correspondem a variações entre 0.0 e 1.7. Também são limitadas as saídas entre 0.0 e 0.7, já que são apenas 8. As *flags* compreendem a variação máxima prevista de 256 itens, variando de 0.0 a 31.7.

Já os argumentos associados aos operadores T e C representam o índice do temporizador ou contador, respectivamente. A linguagem prevê que esse índice varie de 0 a 255, mas deve ser observada a limitação do equipamento que neste caso é de 64 em ambos os casos. Portanto a variação se restringe a valores entre 0 e 63.

Durante a programação as constantes podem ser declaradas de diferentes formas conforme melhor convier ao programador, ou conforme o tipo do recurso que está sendo configurado. As constantes de 16 bits podem ser configuradas das formas seguintes.

- KM: Representação binária, varia de 0000.0000.0000.0000 a 1111.1111.1111.1111.
- KF: Representação em ponto fixo com sinal, varia de -32768 a 32767.

- KH: Representação Hexadecimal, varia de 0000 a FFFF.
- KY: representação em *bytes*, varia de 0.0 a 255.255. Cada um dos 2 algarismos separado pelo ponto é um *byte* (varia de 0 a 255).
- KT: representação definida para configuração do temporizador, varia de 1.0 a 999.3. O algarismo anterior ao ponto representa o tempo que será contado e o algarismo posterior representa um multiplicador. Desta forma, o CLP é capaz de contar desde frações de segundo a horas
- KC: representação definida para configuração do contador, varia de 0 a 999.
- KS: representação tipo texto alfa-numérico.

Outros conceitos importantes são:

- Acumulador1: É um espaço de memória de 16 bits utilizado no carregamento de valores de *preset* do contador, no valor de temporização e em comparações.
- Acumulador2: É um espaço de memória de 16 bits utilizado em comparações.
- RLO: Trata-se da variável do tipo bit que guarda o resultado de lógicas.

4.3.1 Lógicas de bits

As lógicas de bits são lógicas booleanas e sempre trabalham com dois dígitos, 0 ou 1. Os dígitos citados são conhecidos como dígitos binários. No caso de contatos e relés, 0 representa inativo ou não energizado e 1 representa ativo ou energizado. Essas instruções podem operar com os operadores Q (saída), F (flag), I (entrada), T (saída do temporizador) e C (saída do contador). O argumento pode variar de 0.0 a 31.7 dependendo do caso.

Essas lógicas também são utilizadas para carregar um valor em RLO. Sempre que o registrador RLO está vazio a primeira instrução carrega o valor do bit de entrada no RLO. As instruções seguintes realizam a lógica entre a entrada referenciada e o valor do registrador RLO. O resultado é gravado no RLO.

- Lógica *AND* (lógica “E”): instruções “A”, “AN”, “A(“ e “AN(“. Nesta operação para que a saída esteja em estado lógico 1, todas as entradas devem estar

também em 1. Basta uma entrada em 0 para a saída ser 0.

- Lógica *OR* (lógica “OU”): instruções “O”, “ON”, “O(“ e “ON(“. A operação deixa a saída em 1 caso qualquer das entradas esteja em 1. Apenas se todas as entradas estiverem em 0 a saída será zero.
- Lógica *XOR* (lógica “OU exclusivo”): instruções “X”, “XN”, “X(“ e “XN(“. Lógica que ao tratar 2 entradas, a saída será 0 se as entradas forem iguais. A saída é 1 se as entradas forem diferentes.
- Fecha parênteses “)”.

As instruções AN, ON e XN (incluindo as sucedidas por um parênteses) antes de realizar a lógica propriamente dita invertem o valor lido do bit que está sendo processado, desta forma negando a entrada. As instruções “A(“, “AN(“, “O(“, “ON(“, “X(“ e “XN(“ sempre devem terminar com a instrução “)”. Estas instruções permitem descrever a lógica envolvida em ramos de circuitos digitais mais complexos que podem envolver muitas associações.

4.3.2 Atribuições

As instruções de atribuição são comandos que podem alterar o valor de uma variável. Pode ser resultado de uma lógica de bits ou no caso das instruções SET e RESET, podem servir para configurar ou inicializar outros recursos.

- Operador “=”: escreve em uma saída ou flag o resultado de uma operação lógica, ou seja o resultado gravado em RLO.
- Operador “SET”: Caso RLO esteja ativo, se a instrução estiver associada a um operador do tipo Q ou F, simplesmente ela faz com que o argumento receba o valor lógico 1. Se estiver associada a um operador C, configura o contador especificado para contar a partir do valor determinado.
- Operador “RESET”: Caso RLO esteja ativo, se estiver associada a um operador do tipo Q ou F, simplesmente faz com que o argumento receba o valor lógico 0. Se estiver associada a um operador C ou T, zera o recurso fazendo com que assumam seus valores iniciais pré-configurados.

4.3.3 Temporizadores

Há cinco tipos de temporizadores disponíveis. As diferenças ocorrem especialmente no momento em que a *flag* de saída do temporizador é setada. A saída do temporizador pode ser monitorada, utilizando o operador “T” e o índice do temporizador, que varia de acordo com a quantidade de temporizadores disponíveis, em qualquer parte do programa aplicação. Assim o usuário tem a possibilidade de utilizar a temporização para sincronizar processos, garantir tempo mínimo a determinadas ações ou mesmo impedir que outras excedam um tempo máximo.

Os temporizadores podem ser carregados com valores finais de temporização que variam de 0 a 999 com multiplicadores de 10 ms¹⁷, 100 ms, 1 s e 10 s. Isto permite que sejam configurados para temporizar desde 10ms (configurado com constante de 1.0) até 166,5 minutos (configurado com constante de 999.3).

As contagens de tempo ocorrem de forma crescente, iniciando em zero até o valor de *preset*. O valor do *preset* fica protegido durante a temporização, ou seja, não pode ser alterado se não estiver em zero. Portanto se estiver ocorrendo uma temporização, o tempo de estouro só poderá ser alterado se houver um *reset* do temporizador.

Na FIGURA 16 apresenta-se um exemplo de um pequeno programa que configura e utiliza um temporizador com retardo no desligamento configurado para manter a saída ligada por 3 segundos adicionais.

```
//-- Teste do Temporizador
AN I 1.5    // Se botão 1.5 estiver em zero
L  KT 30.1  // Carrega constante de tempo de 3 segundos (30*0,1)
SF T 1      // Configura temporizador para retardo no desligamento
A  T 1
=  Q 0.7    // Carrega na saída 0.7 valor de saída do temporizador T1
//-- Fim do programa
BE
```

FIGURA 16 - Exemplo de programa usando temporizador

- Operador “SP”: temporizador tipo pulso. A saída é ativada no momento em

¹⁷ milissegundos (ms)

que RLO é ativado e o sinal terá uma largura máxima (tempo) igual ao valor do *preset*¹⁸. Quando o registrador RLO é desativado, o temporizador é zerado automaticamente.

- Operador “SE”: temporizador tipo prolongamento de sinal. A saída é ativada no momento em que o registrador RLO é ativado e o sinal terá largura no mínimo igual ao valor do *preset*. O pulso é mantido mesmo havendo oscilação do RLO durante o tempo de *preset*. Quando o registrador RLO é desativado, o temporizador é zerado automaticamente.
- Operador “SD”: temporizador tipo retardo na ligação. A entrada deve permanecer ativa por no mínimo o tempo de *preset*; posteriormente a saída do temporizador é ativada enquanto o registrador RLO mantiver-se ativo. Quando RLO é desativado, o temporizador é zerado automaticamente.
- Operador “SS”: temporizador tipo retardo na ligação com retenção. O RLO deve permanecer ativo por no mínimo o tempo de *preset*, posteriormente a saída do temporizador é ativada. Neste caso, o tempo em que o RLO é ativado é acumulativo, não necessitando estar ativo continuamente. A saída permanece ativa até que seja zerado o temporizador.
- Operador “SF”: temporizador tipo retardo no desligamento. A saída liga juntamente com o temporizador, permanecendo ligado pelo tempo de *preset* após o RLO estar inativo. O temporizador é zerado no momento do *overflow*¹⁹. Na instrução SF, o RLO deve permanecer desativo por, no mínimo, o tempo de *preset* ou a saída permanecerá em 1 continuamente.

4.3.4 Contadores

Os contadores são sensíveis as variações da entrada quando esta vai de zero para um (inativa para ativa). A saída pode ser acessada utilizando o operador C acrescido do índice do contador. As funções *set* e *reset* funcionam de forma especial com os contadores: a primeira pega o valor guardado no acumulador do CLP e carrega como sendo valor de *preset*; o *reset* como se espera zera as *flags* e o valor

¹⁸ Valor pré-definido para o final da temporização.

¹⁹ Estouro do temporizador, momento em que este chega ao valor do *preset*.

de contagem.

As instruções “CU” e “CD”, respectivamente, incrementam e decrementam o valor de contagem. Os contadores são todos do tipo *UP/DOWN*, ou seja, podem ser incrementados e decrementados normalmente sem necessidade de configurações adicionais. Ao ser decrementado e alcançar o valor zero, o contador ativa a sua saída. O mesmo ocorre quando é incrementado e alcançar o valor máximo de 999.

As instruções dependem do valor de RLO para serem chamadas. Na FIGURA 17, apresenta-se um exemplo de uso do contador 5, onde este é inicializado com valor 3 e será incrementado até 999. Quando sua saída for ativada, a *flag* 0.0 também será ativada. A entrada 1.5 comanda o *preset* e a entrada 1.4

```
//-- Teste do Contador
AN I 1.6
CU C 5      // Se entrada 1.6 variar de 0 p/ 1 incrementa C5
AN I 1.5
L KC 3
S C 5      // Se entrada 1.5 estiver em zero carrega preset C5
AN I 1.4
R C 5      // Se entrada 1.4 estiver em 0 reseta C5
A C 5
S F 0.0    // Se a saída do contador estiver em 1 seta flag 0.0
```

FIGURA 17 - Exemplo de uso do contador

pode zerar a contagem.

4.3.5 Instruções de Comparação

As comparações ocorrem entre os conteúdos do “acumulador1” e do “acumulador2”. O acumulador1 é carregado na primeira operação de carga (*load*). A segunda operação de carga automaticamente carrega o acumulador2. Exemplos de comparações são apresentados na FIGURA 18. Nessas instruções, o registrador RLO recebe o resultado da comparação.

- Operador “!=F”: Retorna verdadeiro se o acumulador1 for igual ao

```

//-- Teste de comparações
L   KF 1
L   KC 0
!=F
=   Q  0.0    // Q0.0 recebe resultado de acumulador1 = acumulador2
><F
=   Q  0.1    // Q0.1 recebe resultado de acumulador1 >< acumulador2
>F
=   Q  0.2    // Q0.2 recebe resultado de acumulador1 > acumulador2
>=F
=   Q  0.3    // Q0.3 recebe resultado de acumulador1 >= acumulador2
<F
=   Q  0.4    // Q0.4 recebe resultado de acumulador1 < acumulador2
<=F
=   Q  0.5    // Q0.5 recebe resultado de acumulador1 <= acumulador2

```

FIGURA 18 - Exemplo de uso de comparações

acumulador2.

- Operador “><F”: Retorna verdadeiro se o acumulador1 for diferente do acumulador2.
- Operador “>F”: Retorna verdadeiro se o acumulador1 for maior que o acumulador2.
- Operador “<F”: Retorna verdadeiro se o acumulador1 for menor que o acumulador2.
- Operador “>=F”: Retorna verdadeiro se o acumulador1 for maior ou igual que o acumulador2.
- Operador “<=F”: Retorna verdadeiro se o acumulador1 for menor ou igual que o acumulador2.

Os operadores de comparação não requerem operandos ou argumentos, portanto, formam uma instrução completa.

4.3.6 Instrução de carga

A instrução de carga é utilizada para copiar para o acumulador1 (ACCU1) ou acumulador2 (ACCU2) um valor descrito por uma constante que pode ser KM,

KF, KH, KY, KT, KC ou KS, conforme preferência do usuário ou o tipo de aplicação. Esta instrução não afeta o valor de RLO. O ACCU2 é carregado com o valor da constante sempre que o ACCU1 já estiver ocupado com um valor anteriormente carregado, conforme exemplificado pela FIGURA 19.

- Instrução “L”: Se acumulador1 não estiver com valor carregado, carrega acumulador1 com constante conforme operador e argumento. Se o acumulador2 estiver carregado, carrega acumulador2 com a constante especificada.

L	KC	3	//Carrega ACCU1 com valor 3
L	KT	30.1	//Carrega ACCU2 com equivalente a 3 segundos
L	KY	9.7	//Carrega ACCU2 com o equivalente hexa 0x0907

FIGURA 19 - Exemplos de constantes

4.3.7 Instruções de controle do programa e outras instruções

São instruções utilizadas para parar a aplicação ou sinalizar seu final, bem como para gastar tempo já que o processo de leitura e processamento da instrução é relativamente demorado.

- Instrução “BE”: Fim do bloco, normalmente utilizado em blocos de programas. Os blocos não são suportados nesta versão do CLP. Esta instrução pode ser utilizada para sinalizar final do programa. Instruções desconhecidas (códigos não previstos) fazem o mesmo efeito.
- Instrução “STP”: O programa pára de ser lido e o CLP vai para o estado “parado”. Quando o CLP se encontra em estado parado só pode retornar à atividade após o *reset* do *firmware*.
- Instrução “NOP 0” e “NOP 1”: Sem operação, não executa nenhuma função e não altera valor dos bits de estado. Pode ser utilizada para gastar tempo.

4.4 O firmware

O firmware está escrito em linguagem C. Os diferentes recursos foram divididos em arquivos isolados para tratamento de seus atributos e funções. Na FIGURA 20, os arquivos são representados na forma de um diagrama de classes. Além dos recursos entradas, saídas, *flags*, contadores e temporizadores, há arquivos de apoio como o de tratamento da memória serial. O arquivo “*main*” contém o *loop* principal do programa CLP, onde são realizadas ciclicamente a leitura das entradas, processamento da lógica e escrita das saídas.

Ao energizar a placa, as portas são colocadas em nível lógico 1 instantaneamente e são inicializados os temporizadores do microcontrolador, memória EEPROM, comunicação serial e demais recursos. Em seguida, o CLP entra no laço principal, ou seja, o *scan*.

Em cada *scan* é trocado o estado do LED1²⁰, realizado o procedimento de auto-teste (onde é zerado o *watchdog* do microcontrolador) e lidas as entradas executando uma filtragem digital evitando leitura de ruídos.

20 A placa possui 4 saídas para LED's ou Buzzer's chamadas de LED1, LED2, LED3 e LED4

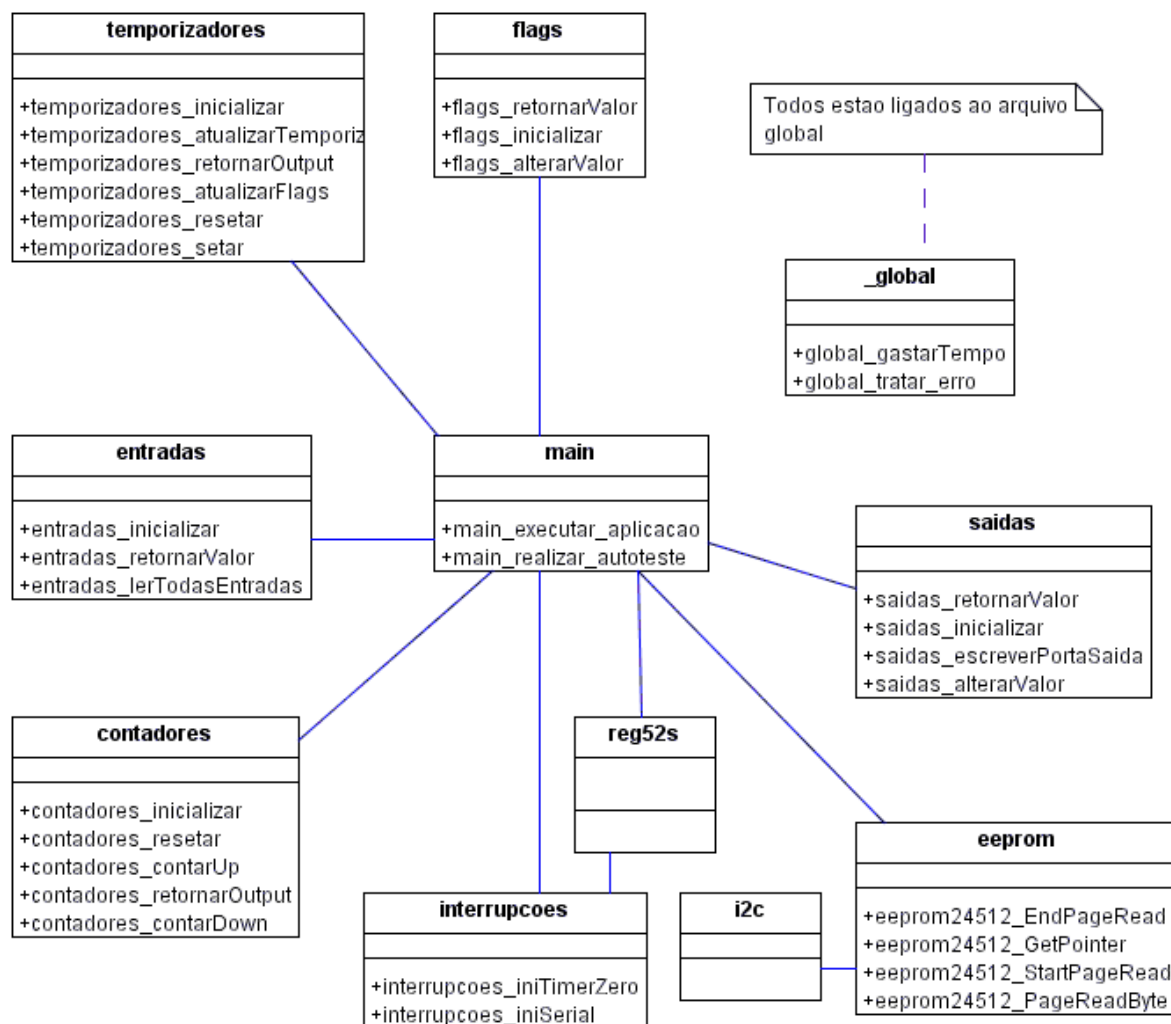


FIGURA 20 - Diagrama de classes do firmware

Um novo estado para cada entrada só será aceito se forem lidos sequencialmente 8 vezes o mesmo valor (0 ou 1). O *firmware* espera 8 bits com mesmo valor em uma variável de 32 bits (essas 8 leituras consecutivas exigem que o valor da entrada permaneça estável por cerca de $100\ \mu s^{21}$). Caso não se consiga obter dados seguros das entradas os valores anteriores são mantidos. Caso ocorra uma falha durante o processamento da lógica as saídas anteriores também são preservadas.

A TABELA 6 mostra três exemplos de seqüências de leitura de uma entrada qualquer. Na primeira linha de dados, sem incidência de ruídos, há a repetição do valor 1 (ativo) da entrada 8 vezes consecutivas o que retorna a leitura bem sucedida da entrada (novo estado = 1). Na segunda linha de dados, uma

²¹ 21 microsegundos (μs)

seqüência de zeros apenas foi aceita após certo ruído no início, após a repetição do valor zero por 8 vezes o novo valor foi aceito (novo estado = 0). Na última linha de dados os ruídos freqüentes impediram a aceitação das leituras, que é desprezada (estado anterior é preservado).

TABELA 6 - Filtro de entrada

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																								1	1	1	1	1	1	1	1
																				0	0	0	0	0	0	0	0	1	0	0	0
1	1	1	1	0	1	1	1	1	0	1	1	0	1	1	1	1	1	1	0	0	1	1	1	0	1	1	1	1	1	1	1

Os temporizadores do CLP são sincronizados com o *scan* através de uma função que atualiza as *flags* apenas quando este procedimento pode ser feito. Isso evita que no meio de um *scan* haja divergência em valores das saídas do temporizadores. A cada 10 ms é chamada a função do temporizador do CLP que atualiza as contagens, internamente são contados 10, 100 ou 1000 estouros no casos das bases de tempo de 100 ms, 1 s e 10 s respectivamente.

4.5 Software montador

O programa de computador permite que o usuário digite a seqüência de instruções que o CLP irá executar. O programa aplicação deve ser digitado através da interface de programação e depois traduzido e transferido ao dispositivo através de uma porta serial.

A montagem é o processo que traduz os códigos digitados pelo usuário em códigos especiais, previamente definidos. O código digitado é sequencial e são esperados operadores e argumentos bem definidos em uma linguagem de mnemônicos booleanos. Durante o processo, a interface (FIGURA 21) mostra ao usuário informações em uma caixa de texto.

O projeto de *software* é composto de três classes principais. As classes “mdiframe” e “mdiedit” são interfaces. A primeira é a janela principal que abre juntamente com o programa. Através da classe “mdiedit” podem ser gerados objetos de edição relativos a diferentes programas aplicação que estão sendo editados. Ambas as classes são interfaces, não possuem funções que armazenam dados e são responsáveis pela ligação entre o usuário e o objeto de montagem.

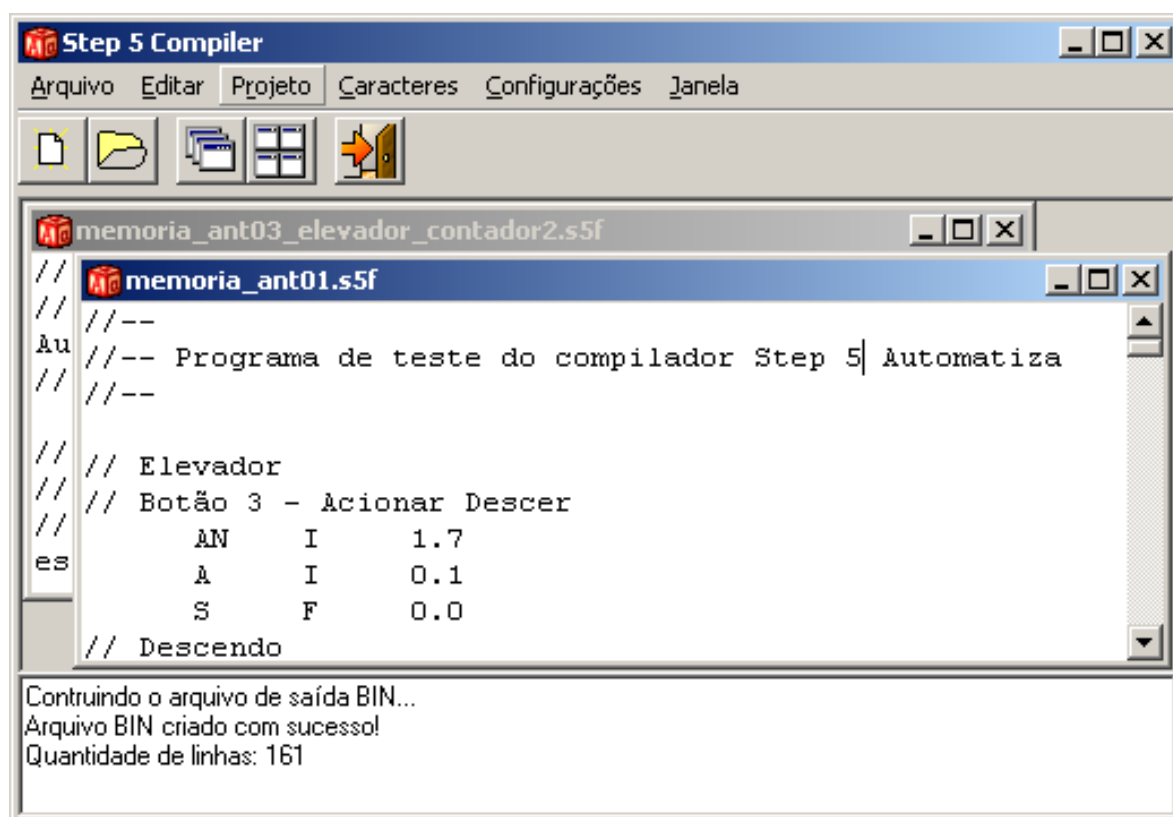


FIGURA 21 - Software montador

A classe “uCompiler” é responsável por processar as informações obtidas do objeto “mdiedit” ativo, gerar o código binário ou retornar mensagens de resposta. Durante a montagem, as instruções são processadas individualmente e as operações têm definidos operadores e argumentos que são conferidos e processados. Em caso de falhas, as mensagens de erro podem levar o usuário a descobrir com maior facilidade onde há problemas no código. A FIGURA 22 mostra o diagrama de classes do *software*.

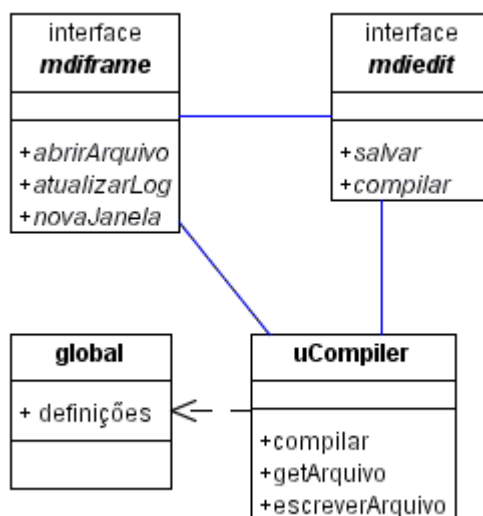


FIGURA 22 - Diagrama de classes do software

4.5.1 Processo de montagem

Ao clicar em “compilar” o usuário, está internamente ativando um processo que irá analisar os comandos digitados linha a linha, como pode ser visto através da FIGURA 23. Durante todas as fases, são verificados os tipos dos dados, se estão dentro da faixa aceitável e se correspondem as operações corretamente. No caso de falhas o processo é interrompido e mensagens de erro são geradas. Durante o processo de montagem, os seguintes passos são executados:

- 1) limpeza da variável que representa o arquivo de saída da montagem, escreve os primeiros *bytes* reservados (fixos) e coloca ponteiro de escrita no início da variável;
- 2) entrada no laço que conta as linhas de texto do programa;
 - 2.1) verifica a operação contida na linha (se for comentário, ou linha vazia ignora) e escreve código binário referente no arquivo de saída;
 - 2.2) verifica o operador analisando se a operação selecionada o suporta e por fim, escreve o código binário;
 - 2.3) verifica o argumento de acordo com o operador, depois, escreve o código binário;
- 3) escreve o conteúdo da variável no arquivo de saída que será o conteúdo da memória EEPROM;

4) escreve no log as mensagens retornadas durante a montagem.

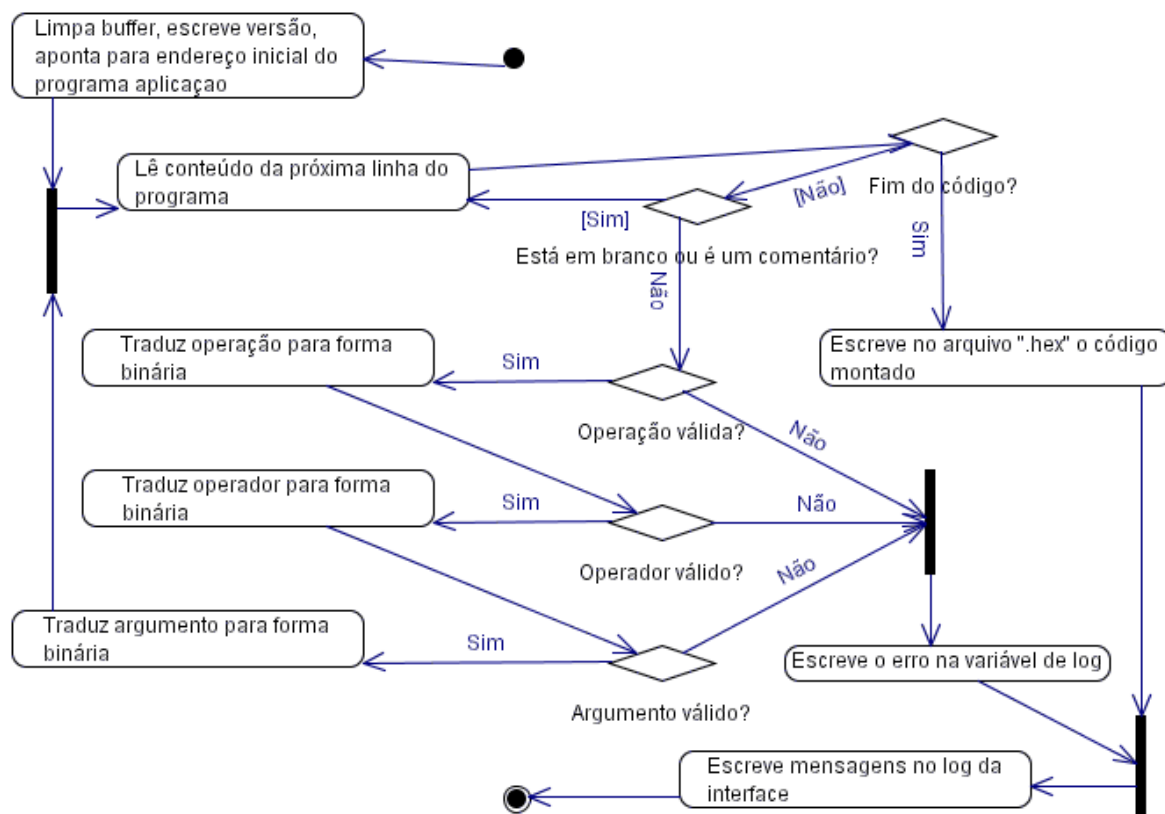


FIGURA 23 - Diagrama de atividades da montagem

Os erros que o montador é capaz de detectar são:

- Operação desconhecida: ocorre quando é digitada uma operação que não é definida no conjunto de instruções do CLP;
- Operando desconhecido (ou não suportado pelo operador): ocorre quando o operador não foi definido no conjunto de instruções levando-se em conta a operação anteriormente analisada;
- Parâmetro fora da faixa aceitável: este erro ocorre quando o usuário tenta utilizar um valor para o argumento que não está dentro da faixa de valores aceitáveis. Um exemplo seria tentar atribuir uma temporização de 1000.0, sendo que o limite é 999.
- Operação ilegal (associação entre operando e argumento não aceitável): algumas instruções apesar de suportar certos argumentos, em alguns casos não suportam determinadas associações. Como exemplo, têm-se uma

operação “=” em “C” (contador). As variáveis do tipo “C” são protegidas internamente, seus valores apenas podem ser lidos;

A FIGURA 24 mostra o software retornando uma mensagem de erro devido ao multiplicador do temporizador estar fora da faixa aceitável. A mensagem informa que na linha 2 do programa-fonte um dos parâmetros (“30” ou “5”) está incorreto. Neste caso o multiplicador (“5”) está fora da faixa que é de 0 a 3.

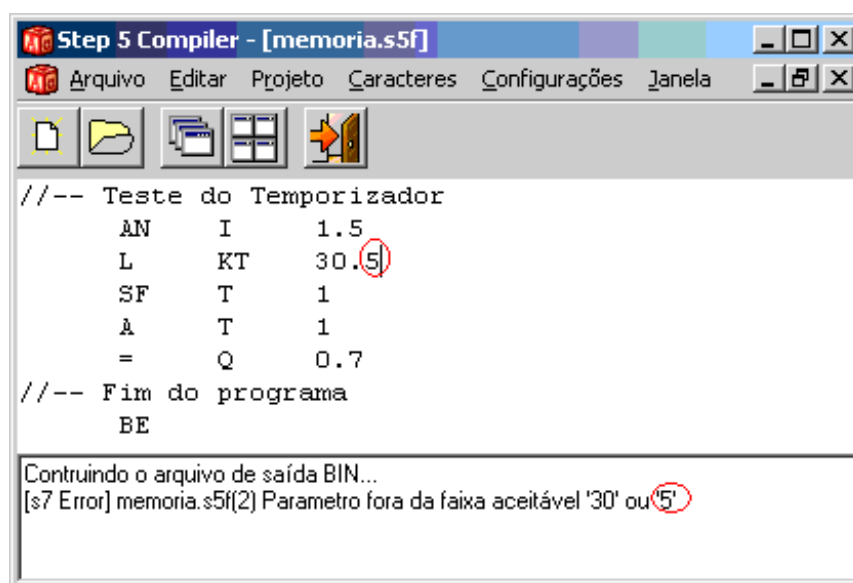


FIGURA 24 - Resposta de erro retornada pelo software

4.6 Carregamento dos comandos no CLP

O carregamento dos comandos é um processo que inicia no *software* onde o usuário escreve o programa na linguagem de lista de instruções que representa a resolução de um problema.

Como demonstrado através da FIGURA 25, ao solicitar o envio dos comandos à placa, o *software* envia os comandos codificados. Para iniciar a transmissão, são trocadas mensagens entre o *software* e a placa para definir que a operação é de carregamento de comandos e informando a formatação dos dados que serão enviados.

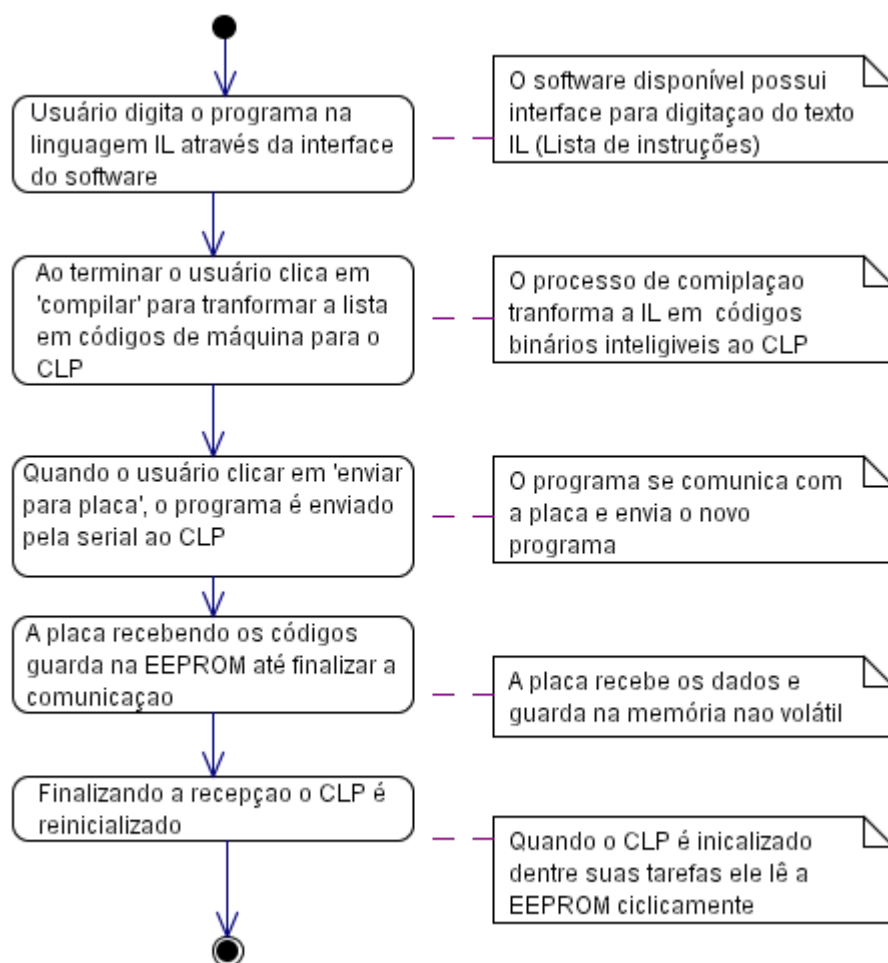


FIGURA 25 - Processo de carregamento de comandos no CLP

Ao iniciar a recepção dos dados, o CLP primeiramente conclui seu ciclo de operação corrente. Os dados correspondentes a comandos são diretamente gravados na memória EEPROM. Ao finalizar a recepção dos dados, o microcontrolador é reinicializado. No processo de inicialização (boot) do CLP são realizadas as inicializações da comunicação serial, memória serial e RAM e protocolo. Em seguida, de forma cíclica, a memória EEPROM será lida e processada normalmente.

Na comunicação entre placa e *software*, a placa é tratada como um dispositivo escravo, ou seja, só envia dados quando requisitada. O *software* por sua vez é o mestre da comunicação.

4.6.1 Comunicação Serial

A placa pode ser conectada ao computador para receber um novo programa aplicação escrito pelo usuário. A conexão serial usa o padrão RS485 que permite que os dispositivos sejam conectados em rede na forma de um barramento linear. Os dispositivos são reconhecidos por endereços únicos na rede. A taxa de transmissão pode variar entre 9,6 kbps e 12 Mbps, sendo que neste projeto a taxa de transmissão adotada é de 57,6 kbps. O cabo utilizado é do tipo par trançado com conectores RJ45.

Até 32 estações mestres ou escravos podem ser conectados a um segmento de rede que pode se estender por até 1.200 m quando a taxa for de 57.600 bps. Nas estações mais distantes entre si, são colocados terminadores no barramento. Em caso de necessidade de expansão do número de estações conectadas ao barramento (mais de 32) ou elevação do limite de distância, podem ser utilizados repetidores para interconexão de diferentes segmentos.

O protocolo de comunicação utilizado é o protocolo MODBUS, onde basicamente as funções utilizadas são de leitura e escrita de registros na memória EEPROM.

A comunicação entre o CLP e o PC deve ser capaz de realizar as seguintes tarefas:

- *handshaking* entre computador e CLP;
- computador recebe identificação do CLP;
- computador envia novo programa ao CLP;
- computador recebe dados gravados na memória do CLP.

No início da comunicação é realizado o *handshaking* entre *software* e *firmware* o que assegura que trata-se da placa correta para onde será transmitido o novo programa aplicação. O *software* então solicita envio de detalhes de identificação da placa especialmente para conhecer a versão para que os dados sejam enviados no formato correto.

O *software* envia uma mensagem informando que dará início à

transmissão de um programa, aguardando aceitação da placa (que pára a execução dos *scans*). Enquanto ocorre a transferência da aplicação, o firmware reescreve a memória EEPROM que contém o programa aplicação. Após enviar um novo programa ao CLP, o computador lê as informações da memória da placa para verificar se o envio foi bem sucedido. Se o novo programa está correto, a transmissão é finalizada, deixando o CLP pronto para ser reinicializado e retomar a execução dos *scans*.

4.7 Aferição e ensaios

Na fase final de desenvolvimento do firmware, a partir de dados obtidos através das folhas técnicas do microcontrolador e da memória serial são realizadas estimativas de desempenho do CLP. Além dos cálculos, para verificar a eficácia no controle realizado pelo CLP nas diversas programações possíveis são realizados ensaios. Estes ensaios são feitos em uma placa conectada a sensores e atuadores simulando uma situação real.

4.7.1 Estimativas de tempo

A TABELA 7 mostra uma planilha contendo cálculos que estimam o tempo do *scan* do CLP. A estimativa é pessimista, ou seja, foram considerados os tempos máximos para realização das tarefas. Esse tempo irá influenciar na precisão dos temporizadores e sensibilidade das entradas. A estimativa baseou-se num programa aplicação com 200 instruções.

Os tempos de execução de instruções do 8051, de tratamento de uma instrução, leitura das entradas e escrita das saídas foram obtidos através de cálculos baseados no clock do microcontrolador. O tempo de leitura da EEPROM, além do clock, leva em consideração o tempo de acesso da memória (obtido através da folha de dados da memória 24LC64). A quantidade média de instruções do 8051 para execução as diversas funções foram obtidos após compilação do *firmware* e análise do arquivo *assembly*²².

²² Neste caso trata-se do assembly do 8051, que é a linguagem de baixo nível de programação do microcontrolador.

Descrição	Valor	Obs
Tempo para ler 1 byte da EEPROM	0,000100	Aproximadamente 100us
Tempo para execução de cada instrução do microcontrolador	0,000001	(Em segundos) O 8051 num clock de 11.0592MHz tem um ciclo de máquina próximo a 1us
Média de instruções para tratamento do comando	100,000000	Quantidade média de instruções de máquina que o 8051 fará para tratar o byte lido que simboliza o comando que o CLP irá executar
Tempo para tratar o comando	0,000100	(Em segundos) Tempo do ciclo de máquina x quantidade de instruções do 8051
Média de instruções para tratamento do argumento	50,000000	Quantidade média de instruções de máquina para tratamento do argumento da instrução do CLP
Tempo para tratar o argumento	0,000050	
Número de passos do CLP	200,000000	Quantidade de instruções que o CLP irá executar
Média de bytes por passo do CLP	3,000000	A maioria das instruções envolve 3 parâmetros (operação, operador e argumento), porém algumas têm apenas 1 (operação) por não necessitarem de argumentos
Quantidade de bytes lidos da EEPROM	600,000000	Número de passos x média de bytes
Tempo gasto para ler entradas	0,010000	(Em segundos)
Tempo gasto na leitura da memória	0,060000	(Em segundos)
Tempo gasto na execução dos comandos	0,020000	(Em segundos)
Tempo gasto na execução dos argumentos	0,010000	(Em segundos)
Tempo gasto na escrita das saídas	0,000100	(Em segundos)
Estimativa de tempo de cada ciclo	0,100100	(Em segundos)

TABELA 7 - Estimativa de tempo do *scan*

O tempo empregado para leitura das entradas e escrita das saídas não varia de acordo com o tamanho da aplicação. No entanto, são tempos relativamente baixos quando comparado com o tempo gasto na leitura da memória serial. A execução dos passos do programa consome cerca de 90% do tempo gasto em cada *scan*. As atividades realizadas nesses passos incluem leitura da memória e execução dos comandos e argumentos.

Em situação normal, os temporizadores tem um erro não acumulativo de “menos uma” contagem, ou seja, se está configurado um estouro para 10 contagens, esse pode ocorrer na nona contagem. Normalmente, portanto, em uma configuração de 250 ms, por exemplo, o estouro do temporizador ocorrerá entre 240 e 250 ms. Nessa situação, o estouro ocorre, portanto, aos 245 ms com uma incerteza de medição de 2% para mais ou para menos.

O resultado final da planilha expõe um tempo de 100,1 ms para o *scan* com 200 instruções de programa. Este tempo é mais de dez vezes superior a menor variação do temporizador que é de 10 ms no “pior caso”. Neste caso a eficiência do temporizador do CLP está comprometida. Com isso o estouro do temporizador

configurado para 250 ms, por exemplo, pode ocorrer entre 240 e 360 ms, elevando a incerteza de medição que era de $\pm 2\%$ para $\pm 25\%$.

A partir das estimativas foram realizados diversos ensaios testando todos os tipos de funções do controlador. Foram comparados tempos estimados com tempos reais de resposta do CLP. Os tempos são fundamentais para o dispositivo pois o número de passos do programa aplicação e a quantidade de recursos utilizados são variáveis de acordo com a necessidade do usuário.

4.7.2 Exemplo de aplicação

As funções desempenhadas pelo CLP neste trabalho foram testadas em uma estrutura que simula um elevador de duas paradas (andares). O motor e o circuito de acionamento são externos ao controlador. Este elevador possui dois botões para chamar o elevador até o respectivo andar e dois sensores para informar que chegou em um determinado andar. Há ainda um motor DC conectado aos relés de saída que fará o acionamento da cabina. Essas conexões são mostradas na FIGURA 26.

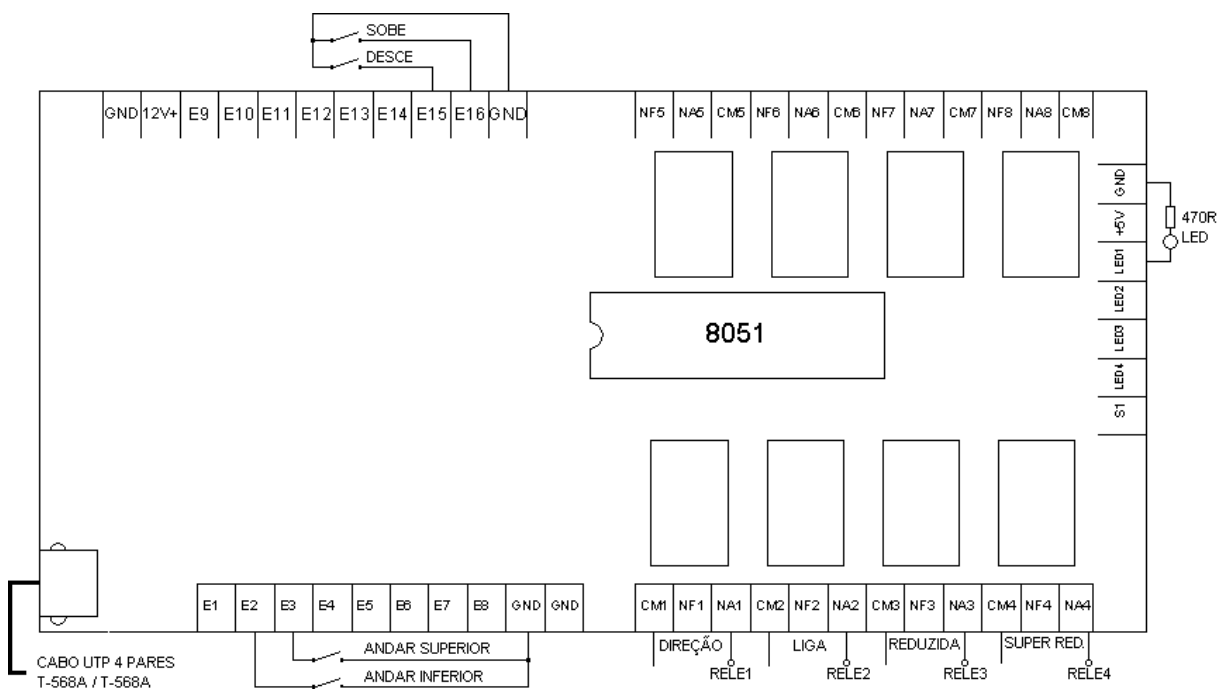


FIGURA 26 - Conexões da placa do CLP para o ensaio do elevador

Conforme esquema da FIGURA 27, quatro relés de saída do CLP são

utilizados para acionamento do motor permitindo três variações de velocidade de deslocamento da cabina: velocidade normal, reduzida e “super reduzida”. A velocidade normal utiliza a onda completa do secundário do transformador “TR1”. A velocidade reduzida aproveita apenas meia onda da saída do transformador. No caso da velocidade “super reduzida”, além de somente aproveitar meia onda, há uma maior redução de tensão já que a corrente passa por três diodos, não apenas um.

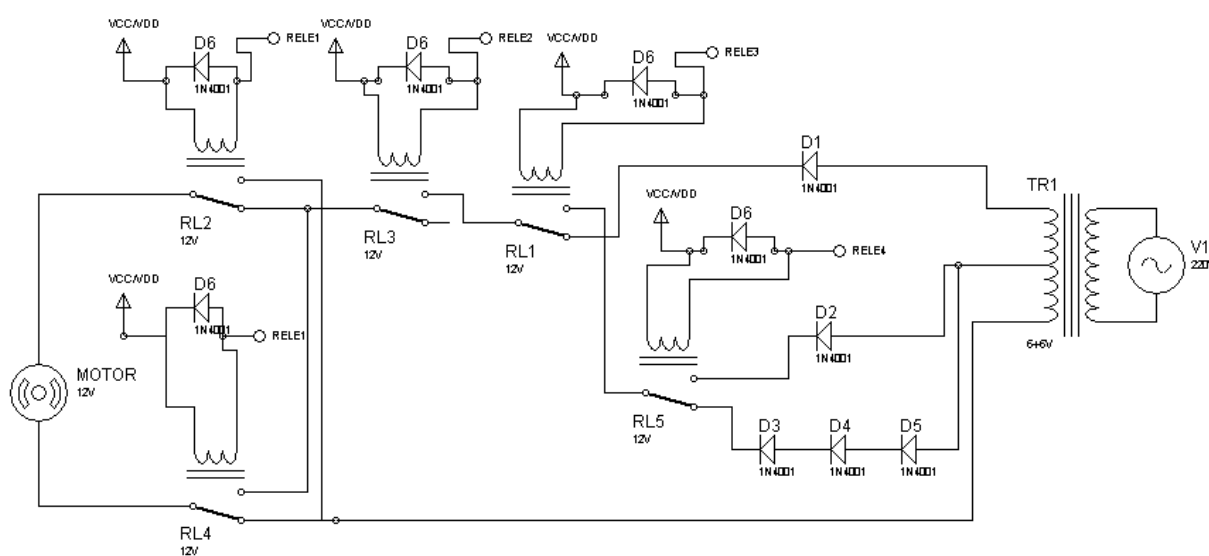


FIGURA 27 - Esquema elétrico de acionamento do motor

O relé 1 do CLP (no esquema chamado RELE1) está conectado a dois relés externos que chaveiam o sentido de rotação do motor. O relé 2 (RELE2) funciona como uma chave liga/desliga, se estiver aberto o motor estará desligado. O relé 3 chaveia a velocidade do motor, se estiver fechado o motor está em velocidade “normal”, se estiver aberto o relé 4 chaveia o motor para a velocidade reduzida ou super reduzida.

A aplicação do elevador, inicialmente realizou aferição das funções e tempos propostos. A estrutura com botões e sensores de paradas foi usada para verificar as lógicas booleanas e a performance do controlador em um programa com 40 passos. A FIGURA 28 mostra um trecho de código do elevador.

```

// Elevador
// Botão 3 - Acionar Descer
// Se bt.3 estiver press. e o elev. não estiver no andar inferior
    AN    I    1.6
    A     I    0.1
    S     F    0.0

// Descendo
// Liga motor + Sentido descida + Velocidade reduzida
    A     F    0.0
    S     Q    0.0    //Desce
    A     F    0.0
    R     Q    0.1    //Liga
    A     F    0.0
    R     Q    0.2    //Velocidade reduzida
    A     F    0.0
    S     Q    0.3    //Não aciona velocidade super reduzida

// Sensor 2 - Parar de descer
// Se acionar sensor do andar inf., zera flag que faz o elev. descer
    A     F    0.0
    AN    I    0.1
    R     F    0.0
    ...

```

FIGURA 28 - Trecho de código do elevador (subida)

Primeiro são analisadas as entradas do botão que chama o elevador para cima e o sensor que informa se o elevador já está no andar de cima. Se o botão está pressionado e o elevador não se encontra nesta posição o *flag* de subida recebe o valor 1 (ativa).

A seguir o código verifica se o *flag* está ativo, se estiver o motor deve ser acionado. Para isso, nesta aplicação é utilizada a velocidade reduzida, o relé de sentido é configurado para subida e o relé de ligação do circuito é acionado (os relés são acionados com valor zero). O relé que coloca o elevador em velocidade super reduzida permanece desativado. A seguir, o elevador verifica o sensor de parada do andar superior, se estiver sinalizando a chegada da cabina, o *flag* de subida é zerado para que posteriormente o motor seja desligado eletricamente (FIGURA 29). O código completo da aplicação do elevador está no APÊNDICE C.

```
// Parar Elevador
// Se ambas as flags (subida e descida) estão zeradas pára o motor
AN    F    0.0
AN    F    0.1
S     Q    0.0
AN    F    0.0
AN    F    0.1
S     Q    0.1
AN    F    0.0
AN    F    0.1
S     Q    0.2
AN    F    0.0
AN    F    0.1
S     Q    0.3
```

FIGURA 29 - Trecho de código do elevador (parada)

5 METODOLOGIA

5.1 Métodos Aplicados

O tema CLP é amplamente estudado e aprofundado por diversos autores em instituições de ensino e por empresas de desenvolvimento de equipamentos eletrônicos. Os livros com grande carga teórica são base para entendimento amplo e geração dos textos conceituais deste trabalho. Algumas apostilas e livros didáticos foram pesquisados e trazem uma abordagem mais prática do dispositivo, contribuindo para entendimento das aplicações do CLP. Por fim, documentos gerados por fabricantes de CLP's são importantes para conhecimento da arquitetura interna e para o desenvolvimento do produto nos moldes de modelos já estabelecidos no mercado.

Muitos autores, inclusive escrevendo em língua portuguesa, publicaram livros no tema. Muitos desses livros abordam o CLP de forma científica e teórica mostrando sua estrutura interna padrão, forma de funcionamento, de tratamento das instruções, além de introduzirem as linguagens de programação, protocolos de comunicação e teoria de classificação dos CLP's. Com pequenas ressalvas, os autores convergem sobre a estrutura e funcionamento básico do CLP. Há divergências nos métodos de classificação do CLP, os fabricantes acompanham essa tendência também categorizando seus produtos de formas diferentes.

Outros autores, ligados a instituições de ensino, publicaram livros e apostilas abordando o CLP de forma mais prática. As principais publicações, apesar de serem aplicáveis em diversas áreas e situações, mostram aplicações na área de automação industrial, pioneira e principal empregadora desse dispositivo. Alguns desses documentos são estudados e contribuem para melhor entendimento do CLP nas formas que este pode ser aplicado. As apostilas também normalmente se baseiam em um produto específico, o que por vezes é interessante para uma compreensão menos ampla, porém mais profunda.

Os fabricantes já estabelecidos através, de manuais e folhas de dados,

geraram documentos que são importantes especialmente para definição dos detalhes internos do dispositivo. A idéia é desenvolver um CLP que não cause impacto e problemas de adaptação aos projetistas, então é importante conhecer os detalhes mais específicos sobre os produtos que servem de inspiração para este trabalho.

O trabalho conta, primeiro, com uma pesquisa bibliográfica utilizando os livros mais abrangentes sobre o assunto a fim de dar embasamento geral sobre o tema e nortear os passos seguintes. A pesquisa bibliográfica, segundo Lakatos e Marconi (2003, p.183), “propicia o exame de um tema sob novo enfoque ou abordagem, chegando a conclusões inovadoras.”

Após uma fase essencialmente teórica, a pesquisa segue direcionada a documentos que expõem o CLP de forma aplicada a fim de definir os detalhes internos da implementação. Nesta etapa, a pesquisa ocorre sob demanda das etapas de desenvolvimento, ou seja, os documentos são buscados conforme surgem dúvidas na implementação. Inicia-se, portanto, a chamada pesquisa experimental, que, para Gil (2007, p.47), “consiste em determinar um objeto de estudo, selecionar as variáveis que seriam capazes de influenciá-lo, definir as formas de controle e de observação dos efeitos que a variável produz no objeto.”

O local para desenvolvimento deste projeto é a sede da empresa Automatiza, sob orientação do diretor do setor de desenvolvimento, Sr. Eduardo Guse e pelos orientadores acadêmicos. O conjunto de requisitos foi elaborado pelo diretor administrativo da empresa Automatiza, Sr. Juliano Cypriano Garcia, que pode ser considerado o “dono do projeto”, por ser o responsável por receber o projeto e dar o seu aval.

A formação do projeto informacional, que identifica o problema, as possíveis soluções e especifica uma delas, antecedeu este trabalho. A Automatiza ao ser solicitada por seus clientes para elaborar diversas soluções, viu o CLP como melhor alternativa para atender muitos casos. A partir disso, gerou uma listagem de requisitos de projeto. Este trabalho parte dos requisitos estabelecidos, iniciando etapas de desenvolvimento conforme a TABELA 8:

TABELA 8 - Etapas do trabalho

Nº	Nome etapa	Descrição	Produto
1	Pesquisa	Pesquisa sobre as tecnologias aplicadas nos CLP's comerciais	Resultado da pesquisa
2	Projeto	Desenvolvimento do projeto de desenvolvimento do produto	Projeto de produto
3	Firmware Etapa 0	Criar estrutura do programa com leitura e escrita das entradas e saídas e da memória EEPROM e desenho da placa no simulador	Firmware preliminar
4	Firmware Etapa 1	Desenvolver interpretação das instruções de lógica booleana, contagem e temporizações	Firmware final
5	Software	Desenvolver software montador	Software
6	Testes	Testes e ajustes do protótipo	Protótipo
7	Documentação	Redação da monografia	Monografia e Apresentação

1. Pesquisa sobre as tecnologias aplicadas nos CLP's comerciais para confrontamento com os requisitos estabelecidos pelo cliente. Basear-se em outros produtos é importante para seguir um padrão já existente e ajuda a estabelecer as demais características do CLP.
2. Elaboração do projeto de desenvolvimento do produto. Etapa em que é gerado o projeto conceitual onde é feita a engenharia de *hardware* e *software*. Na análise do *hardware* é verificado se a placa disponível pode atender ao projeto. Na engenharia de *software* é feita a modelagem e o projeto do programa de computador.
3. Criar estrutura do programa com leitura e escrita das entradas, saídas e memória EEPROM e desenho da placa no simulador. Estágio inicial de programação em que é montada a base do código, facilitando inserção de funções que utilizarão esses recursos.
4. Desenvolver interpretação das instruções de lógica booleana, contagem e temporizações. Este estágio é de grande aprofundamento no CLP, onde são

conhecidas as instruções e implementados códigos para tratamentos das variáveis e leitura de um código gravado na memória de maneira manual.

5. Desenvolver *software* montador. Esta etapa visa implementação do *software* editor da lista de instruções e tradutor, o que irá gerar os códigos que até a etapa anterior, em testes, estavam sendo inseridos manualmente.
6. Testes e ajustes do protótipo. Fundamental estágio de refinamento e validação da solução. O *firmware* é gravado e posto na placa para realização dos ensaios que envolvem além de testes, as análises de desempenho, verificando se o sistema suporta as instruções e recursos propostos.
7. Redação da monografia.

5.2 Apresentação, análise e discussão dos resultados

Para elaborar o projeto são associados os requisitos do cliente com as definições sugeridas pela norma IEC 6113. Como resultado obtêm-se um produto utilizando uma linguagem de programação já estabelecida embarcado em um gabinete definido pela literatura como *frame open* e categorizado como um microCLP (considerando funcionalidades, quantidade de entradas e saídas e custo). O protocolo de comunicação utilizado no projeto (MODBUS) também respeita a premissa da utilização de padrões, já que é o mais amplamente utilizado no ambiente industrial (INTELLICOM, 2007).

A linguagem C para o *firmware* e C++ para o *software* são escolhidas por facilitarem a manutenção do código e por já haver bastante produção de código realizada pela própria empresa Automatiza. No *firmware*, por exemplo, os arquivos de tratamento da memória serial e de implementação do protocolo MODBUS foram baseados em arquivos já desenvolvidos e testados em outros projetos anteriores. Isso reduziu a carga de trabalho e trouxe maior confiabilidade ao projeto.

A primeira etapa de desenvolvimento a ser executada é de elaboração do *firmware*. Inicialmente são implementadas funções simples de lógicas de bits, evoluindo para contagens, comparações, etc. Tudo é implementado e simulado através de um *software* simulador. Na fase inicial, é gravado um microcontrolador e

testado na placa para garantir que as respostas simuladas estejam corretas. A partir daí apenas na fase final de testes são feitas novas simulações via *software* e definitivamente a base de testes fica sendo a placa final.

No *software* simulador utilizado não há disponível o membro 89C51ED2 da família 8051, assim o projeto tem de ser simulado utilizando o microcontrolador 80S52 que é de menor capacidade. Devido à necessidade de utilização de um dispositivo com menor capacidade, a simulação é feita reduzindo recursos do CLP (A quantidade de temporizadores baixou de 64 para apenas 2). Por realizar simulações utilizando elementos diferentes dos utilizados na placa houve uma elevação de riscos de insucesso.

De fato, ao testar na placa, com a disponibilização de 64 temporizadores surgem problemas que causam travamento do CLP. A lógica de disparo das temporizações e atualizações das contagens do CLP tem que ser refeita tomando cuidado com tempos mínimos necessários para rodagem das interrupções internas no microcontrolador e respeitando a precisão dos temporizadores de CLP.

Os resultados obtidos nos ensaios finais são satisfatórios. O CLP rodando um programa com 200 instruções leva cerca de 100 ms para realizar um *scan*. No caso de um programa como esse, o usuário deve ter o cuidado de não utilizar o sistema em situações críticas que requerem temporizações na ordem de centésimos de segundos. Para um programa de 200 instruções, o CLP terá seu desempenho satisfatório apenas nas temporizações da ordem de décimos de segundos.

A TABELA 9 mostra uma breve comparação entre alguns aspectos do CLP deste projeto e outros disponíveis no mercado (todos os dispositivos comparados são microCLP's). Os exemplos são dispositivos comerciais fabricados por empresas consagradas no ramo. Essas empresas possuem extensa linha de CLP's disponíveis. Os campos em branco representam informações desconhecidas devido a dificuldade de cruzar dados.

O fato dos fabricantes não utilizarem um padrão para apresentação das características dos produtos dificulta bastante a comparação entre controladores lógicos. É o caso do modelo AB 1760-L12DWD da empresa Allen-Bradley que informa o tamanho da memória através do número de instruções suportadas. No

caso do CLP aqui projetado cada instrução ocupa uma média de 3 *bytes*, logo a memória suporta cerca de 2700 instruções.

TABELA 9 - Comparativo entre o CLP deste projeto e outros disponíveis no mercado

CLP / CARACTERÍSTICA	Automatiza 16FO8-S5	AB 1760-L12DWD Pico Controller	Siemens SIMATIC S7-200 CPU222	Triangle T100MD888+
Tamanho da memória	8 KB	512 Instruções	4 KB	12 KB
Alimentação	12VDC	12VDC		
Contadores	64	16	256	
Temporizadores	64	16	256	
Flags	256	16	256	
Entradas digitais	16 integradas	8 integradas	8 integradas, 40 máximo	8 integradas
Saídas	8 relés	2	6 integradas, 38 no máximo	8 integradas
Entradas/saídas analógicas	0/0	0/4 relés	No máximo 8/2 ou 0/4	4/1
Corrente de saída	1A	8A resistiva e 3A indutiva		6x1A e 2x10A
Comunicação	1 porta RS485, Modbus		AS, Profibus, Ethernet, Internet e modem	1 porta RS232, 1 porta RS485, MODBUS, OMRON C20H, EMIT 3.0

Fontes: <http://www.ab.com/>, <http://www.siemens.com/>, www.tri-plc.com/

Pelos dados apresentados, as variações entre os modelos comparados são bastante significativas, o que exemplifica a riqueza de produtos disponíveis no mercado. No item entradas e saídas analógicas o CLP aqui projetado mostra certa desvantagem.

6 CONSIDERAÇÕES FINAIS

A idéia de desenvolver o Controlador Lógico Programável era um antigo desejo da empresa Automatiza. O projeto representa uma importante evolução na apresentação de soluções a seus clientes e na produção de diferentes produtos utilizando uma mesma matriz (placa, microcontrolador, sistema de comunicação e até *software*).

O tema CLP é amplamente explorado; muitos livros de qualidade, especialmente em língua inglesa, oferecem informações sobre o assunto. As instituições de ensino também oferecem disciplinas que tratam o assunto, especialmente os cursos de automação que exploram o CLP como peça chave em linhas de produção e montagem de indústrias modernas. Outras aplicações que crescem juntamente com a evolução dos CLP's estão na automação predial e residencial, além da utilização e sistemas de monitoramento de diversos dados obtidos através de transdutores.

A metodologia empregada foi eficaz. As etapas do projeto foram respeitadas, os prazos, dentro de determinada tolerância, foram cumpridos. Apenas no decorrer do trabalho algumas dificuldades na simulação e nos ensaios causaram pequenos atrasos. Essas dificuldades estão relacionadas ao microcontrolador utilizado, que possui documentação pouco detalhada e não é suportado pelo *software* de simulação utilizado.

Pode-se considerar que todos os objetivos do projeto foram alcançados. O universo do controle lógico foi reduzido, detalhes foram abstraídos e buscou-se extrair os princípios básicos em termos de funcionalidades e arquitetura. Desta forma o escopo foi simplificado a tal ponto que tornou possível a conclusão bem sucedida do trabalho. A aceitação da empresa Automatiza foi positiva, já se vislumbram aplicações e novos investimentos em desenvolvimento de melhorias neste projeto.

7 RECOMENDAÇÕES PARA TRABALHOS FUTUROS

O projeto utilizando placa única pode evoluir agregando as seguintes melhorias: possibilitar utilização em rede ethernet; durante o processo de auto-teste deve ser verificado e tratado falhas caso as saídas sejam diferentes das imagens; possibilitar expansão dos recursos de temporização e contagens; utilizar relógio de tempo real e criação de instruções para acionamento de acordo com a hora do dia e o dia da semana; gerar eventos e enviar ao computador ou à IHM; integrar com outros produtos e criar funções para diagnóstico do CLP.

O projeto também pode evoluir na direção dos grandes CLP's modulares com a criação de barramento de conexão, melhorias em conectividade e escalabilidade, com possibilidade inclusive de expansão das entradas e saídas, memória, distribuição de processamento, etc.

O escopo reduzido deste trabalho deixou muitas possíveis melhorias para o *software* em aberto, tais como: aceitar outras linguagens de programação como *ladder*, *grafcet*, texto estruturado e diagrama em blocos; recepção de eventos; simulação do programa aplicação antes da gravação e ferramentas visuais de diagnóstico do CLP.

BIBLIOGRAFIA COMPLEMENTAR

AHMED, Asfaq. *Eletrônica de potência*; tradução Bazán Tecnologia e Lingüística; revisão técnica João Antonio Martino. São Paulo: Prentice Hall, 2000.

BARBI, Ivo. *Eletrônica de potência*. 3. ed. Florianópolis: Editora do autor, 2000.

GIMENEZ, Salvador P. *Microcontroladores 8051*. São Paulo: Pearson Education do Brasil, 2002.

HUBBARD, John R. *Teoria e Problemas de Programação em C++*. 2.ed. Porto Alegre: Bookman, 2003. ISBN 0071353461

NICOLOSI, Denys Emílio Campion. *Microcontrolador 8051 com linguagem C: prático e didático – família AT89S8252 Atmel*. São Paulo: Érica, 2005.

OGATA, Katsuhiko. *Engenharia de controle moderno*. 2.ed. Rio de Janeiro: Prentice Hall do Brasil, c1993. 781p. ISBN 8570540450

PETERS, James F.; PEDRYCZ, Witold. *Engenharia de Software*. Rio de Janeiro: Elsevier, 2001. ISBN 8535207465

SIEMENS; *Programming with STEP 7 V5.2*. Nuernberg – Alemanha: 2002.

SIEMENS; *Statement List (STL) for S7-300 and S7-400 Programming*. Nuernberg – Alemanha: 2002.

VILLAÇA, Marco. *Introdução aos microcontroladores*. Florianópolis: Mimeo, 2007.

REFERÊNCIAS

ATMEL. Disponível em <http://www.atmel.com> Acesso em 02/10/2007.

BRYAN, L.A. *Programmable controllers: theory and implementation*. Atlanta: Industrial Text Company, 1997. ISBN 0-944107-32-X

CEFETSC. *Normas de apresentação do trabalho de conclusão de curso – TCC – Curso Superior de Tecnologia em Sistemas Digitais*. Florianópolis: 2007.

CORRADI. *Controladores Lógicos Programáveis*. 2005.

GARDINI, Giacomo; LIMA, Norberto de Paula. *Dicionário de Eletrônica*. 3.ed. Hermus, 2003.

GEORGINE, Marcelo. *Automação Aplicada Descrição e Implementação de Sistemas Sequenciais com PLCs*. São Paulo: Érica, 2000. ISBN 85-7194-724-4

GIL, Antonio Carlos. *Como elaborar projetos de pesquisa*. 4.ed. São Paulo: Atlas, 2007.

HACKWORTH, John R.; HACKWORTH Jr., Frederick D. *Programmable Logic Controllers: Programming Methods and Applications*. Prentice Hall, 2003. ISBN 0130607185.

IEC61131. Disponível em <http://www.iec61131.com.br> Acesso em 13/11/2007.

INDUSTRIAL Text and Video co; *Glossary of Terms used in Programmable Controller-based System*. 2002.

INTELLICOM Innovation. *Modbus*. Disponível em <http://www.intellicom.se/modbus.shtml>. Acesso em 20/11/2007.

JACK, Hugh. *Automating Manufacturing Systems with PLCs*. Versão 4.5: e-book, 2004.

LAKATOS, Eva Maria; MARCONI, Marina de Andrade. *Fundamentos de Metodologia Científica*. 5. ed. São Paulo: Atlas, 2003.

MODBUS. *MODBUS over serial line specification and implementation guide V1.0*, 2002.

MODBUS. *MODBUS Application Protocol Specification V1.1*, 2004.

PARR, E.A. *Programmable Controllers an engineer's guide*. 3. ed. Oxford - Grã Bretanha: Elsevier, 2003. ISBN 0 7506 5757 X

REYNDERS, Deon. *Practical Industrial data communications: best practice techniques*. Burlington -USA : Elsevier, 2005. ISBN 0 7506 6395 2

SIEMENS, Simatic S5 - STEP 5 Version 6.6 Manual. Nuernberg – Alemanha:1996.

SILVEIRA, Paulo Roberto da; SANTOS, Winderson E. dos. *Automação e Controle*

Discreto. São Paulo: Érica., 1998.

SOUZA, Marco A. B. *Estudo comparativo entre aplicação de sistemas dedicados e a utilização de controladores lógico-programáveis na automação de sistemas prediais*. 2004, 127 f. Dissertação (Mestrado em Engenharia) – Escola Politécnica da Universidade de São Paulo, São Paulo, 2004.

WEBER, Raul Fernando. *Fundamentos de Arquitetura de Computadores*. 3. ed. Porto Alegre: Sagra Luzzatto, 2004.

APÊNDICES

APÊNDICE A – Estudo das instruções

LINGUAGEM – STEP5			COMPILADO			
OPERADOR	OPERANDO	ARGUMENTO	OPERADOR	OPERANDO	ARGUMENTO	N BYTES
A	(*4)	0.0 a 31.7 (*2)	0x01	0 a 0x05	0 a 0xFF	3
AN	(*4)	0.0 a 31.7 (*2)	0x02	0 a 0x05	0 a 0xFF	3
O	(*4)	0.0 a 31.7 (*2)	0x04	0 a 0x05	0 a 0xFF	3
ON	(*4)	0.0 a 31.7 (*2)	0x05	0 a 0x05	0 a 0xFF	3
X	(*4)	0.0 a 31.7 (*2)	0x26	0 a 0x05	0 a 0xFF	3
XN	(*4)	0.0 a 31.7 (*2)	0x27	0 a 0x05	0 a 0xFF	3
=	(*6)	0.0 a 31.7 (*2)	0x0A	0 ou 0x01	0 a 0xFF	3
A(	-	-	0x03	-	-	1
O(	-	-	0x06	-	-	1
X(	-	-	0x28	-	-	1
)	-	-	0x07	-	-	1
S	(*6)	0.0 a 31.7 (*2)	0x08	0 ou 0x01	0 a 0xFF	3
R	(*6)	0.0 a 31.7 (*2)	0x09	0 ou 0x01	0 a 0xFF	3
STP	-	-	0x22	-	-	1
SP	(*3)	T0 a T255	0x0E	-	0 a 0xFF	2
SE	(*3)	T0 a T255	0x0F	-	0 a 0xFF	2
SD	(*3)	T0 a T255	0x10	-	0 a 0xFF	2
SS	(*3)	T0 a T255	0x11	-	0 a 0xFF	2
SF	(*3)	T0 a T255	0x12	-	0 a 0xFF	2
CU	(*3)	C0 a C255	0x13	-	0 a 0xFF	2
CD	(*3)	C0 a C255	0x14	-	0 a 0xFF	2
!=F	(*3)	-	0x15	-	-	1
><F	(*3)	-	0x16	-	-	1
>F	(*3)	-	0x17	-	-	1
>=F	(*3)	-	0x18	-	-	1
<F	(*3)	-	0x19	-	-	1
<=F	(*3)	-	0x1A	-	-	1
L	(*5)	K0.0 a K999.3	0x0B	(*1)	0 a 0xFFFF	3
LD	(*5)	K0.0 a K999.3	0x0C	(*1)	0 a 0xFFFF	3
T	(*5)	K0.0 a K999.3	0x0D	(*1)	0 a 0xFFFF	3
JU	?		0x1B			
JC	?		0x1C			
CD	?		0x1D			
G	?		0x1E			
BE	?		0x1F			
BEU	?		0x20			
BEC	?		0x21			
NOP 0			0x23			
NOP 1			0x24			
BLD	?		0x25			

(*1) As diferentes maneiras de declarar constantes são diretivas de compilação, o dado é gravado no CLP sempre da mesma forma

(*2) 0.0 a 31.7 equivale a $32 * 8$ pontos = 256

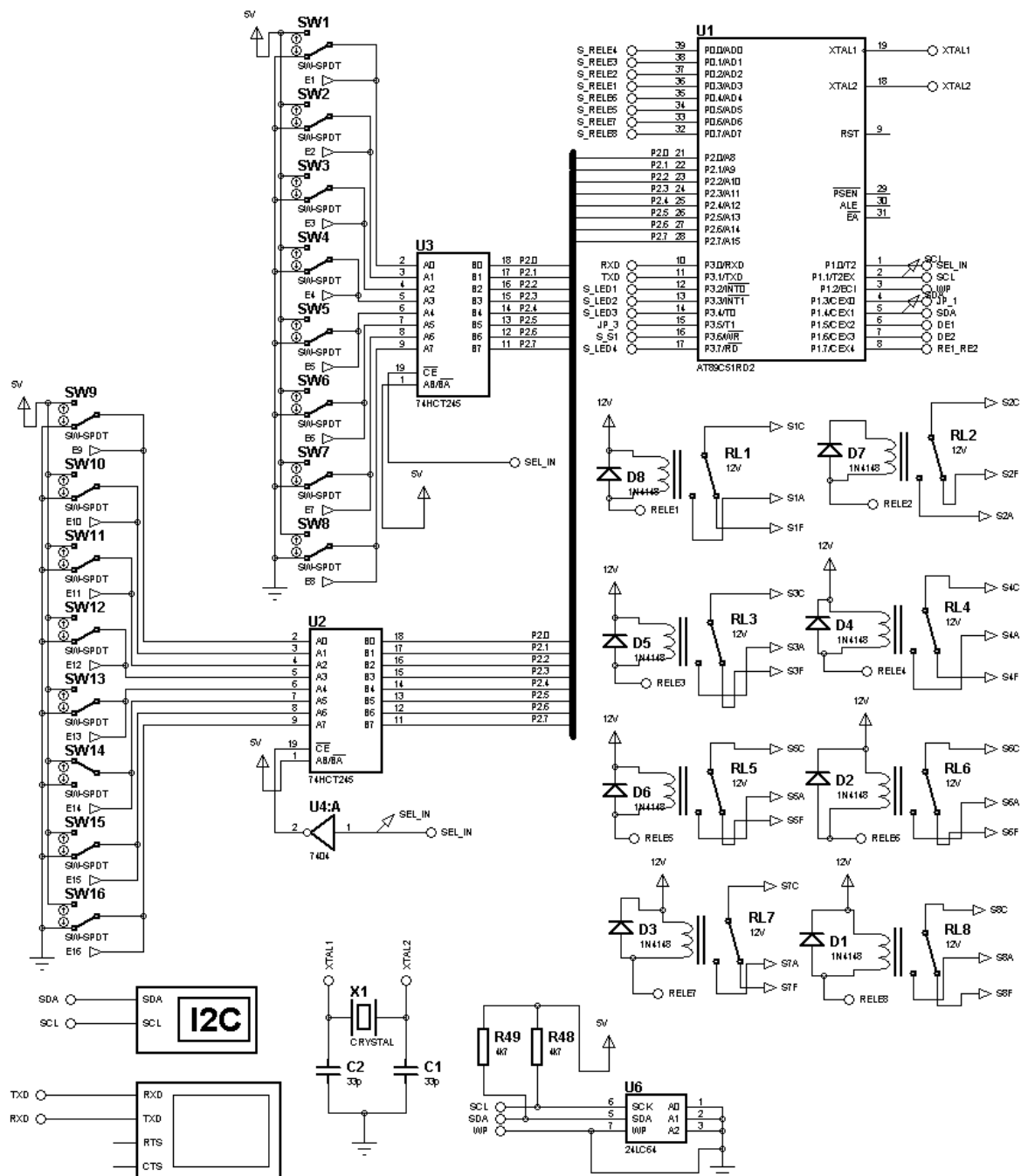
(*3) Instruções que funcionam de acordo com uma sequência de L (load) e comparação, os valores portanto já se encontram carregados em variáveis internas

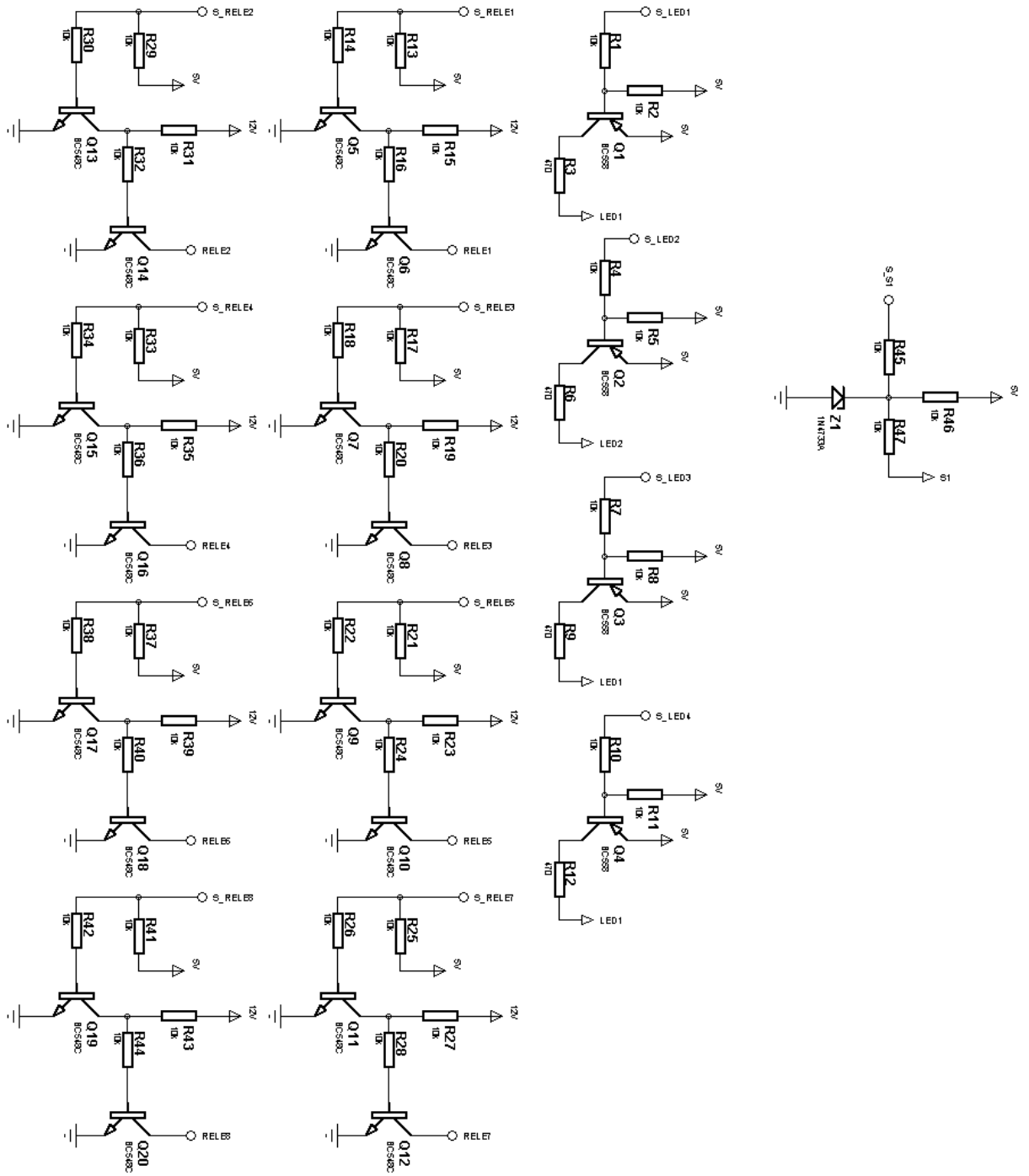
(*4) Q = saída = 0x00, F = relé de controle = 0x01, I = entrada = 0x02, T = status do temporizador = 0x03, C = status do contador = 0x04, SR = relés especiais = 0x05

(*5) As constantes podem ser dos tipos: KM (binária 16bits), KH (Hexa 4 dígitos), KY (2 bytes), KF (ponto fixo -32768 a 32767), KS (Alfanumérica de até 2 char), KT (tempo em BCD de 1.0 à 999.3) e KC (contagem BCD de 0 a 999)

(*6) Q = saída = 0x00, RC = relé de controle = 0x01

APÊNDICE B – Esquemático da placa





APÊNDICE C – Programa do elevador

```
// Elevador
// Botão 3 - Acionar Descer
// Se botão 3 estiver pressionado e o elevador não estiver no andar inferior
    AN    I    1.6
    A     I    0.1
    S     F    0.0
// Descendo
// Liga motor + Sentido descida + Velocidade reduzida
    A     F    0.0
    S     Q    0.0    //Desce
    A     F    0.0
    R     Q    0.1    //Liga
    A     F    0.0
    R     Q    0.2    //Velocidade reduzida
    A     F    0.0
    S     Q    0.3    //Não aciona velocidade super reduzida
// Sensor 2 - Parar de descer
// Se acionar sensor do andar inferior, zera flag que faz o elevador descer
    A     F    0.0
    AN    I    0.1
    R     F    0.0
//Botão 3 - Acionar Subir
    AN    I    1.7
    A     I    0.2
    S     F    0.1
// Subindo
// Liga motor + Sentido subida + Velocidade reduzida
    A     F    0.1
    R     Q    0.0    //Sobe
    A     F    0.1
    R     Q    0.1    //Liga
    A     F    0.1
    R     Q    0.2    //Velocidade reduzida
    A     F    0.1
    S     Q    0.3    //Não aciona velocidade super reduzida
// Sensor 2 - Parar de Subir
// Se acionar sensor do andar superior, zera flag que faz o elevador subir
    A     F    0.1
    AN    I    0.2
    R     F    0.1
// Parar Elevador
// Se ambas as flags (de subida e descida) estão zeradas pára o motor
    AN    F    0.0
    AN    F    0.1
    S     Q    0.0
    AN    F    0.0
    AN    F    0.1
    S     Q    0.1
    AN    F    0.0
    AN    F    0.1
    S     Q    0.2
    AN    F    0.0
    AN    F    0.1
    S     Q    0.3
//-- Fim do programa
BE
```

ANEXOS

ANEXO A – Códigos de funções públicas

				Function Codes		(hex)	Section
				code	Sub code		
Data Access	Bit access	Physical Discrete Inputs	Read Discrete Inputs	02		02	6.2
		Internal Bits Or Physical coils	Read Coils	01		01	6.1
			Write Single Coil	05		05	6.5
			Write Multiple Coils	15		0F	6.11
	16 bits access	Physical Input Registers	Read Input Register	04		04	6.4
			Read Holding Registers	03		03	6.3
		Internal Registers Or Physical Output Registers	Write Single Register	06		06	6.6
			Write Multiple Registers	16		10	6.12
			Read/Write Multiple Registers	23		17	6.17
			Mask Write Register	22		16	6.16
			Read FIFO queue	24		18	6.18
	File record access	Read File record		20	6	14	6.14
		Write File record		21	6	15	6.15
		Read Exception status		07		07	6.7
	Diagnostics	Diagnostic		08	00-18,20	08	6.8
		Get Com event counter		11		0B	6.9
		Get Com Event Log		12		0C	6.10
		Report Slave ID		17		11	6.13
		Read device Identification		43	14	2B	6.21
	Other		Encapsulated Interface Transport	43	13,14	2B	6.19

Fonte: MODBUS

ANEXO B – Conjunto completo de instruções STEP5

Instruction Type	S5	Instruction Type	S5	Instruction Type	S5
Accumulator instructions	TAK, ENT, I, D, ADDBN, ADDKF, ADDDH	Integer math instructions	+F, -F, xF, :F, +D, -D	Logic control instructions, jump	JU, JC, JN, JZ, JP, JM, JO, JOS, JUR
Address register instructions / Register instructions	MA1, MBR, ABR, MAS, MAB, MSB, MSA, MBA, MBS, TSG, LRB, LRW, LRD, TRB, TRW, TRD	Floating-point math instructions	+G, -G, xG, :G	Block instructions	JU, JC, DOU, DOC, BE, BEU, BEC
Bit logic instructions	A, AN, O, ON, A(, O(,), O, S, R, RB, RD= TB, TBN, SU, RU	Comparison instructions	!=F, ><F, >F, <F, >=F, <=F, !=D, ><D, D, <D, >=D, <=D, !=G, ><G, >G, <G, >=G, <=G	Command output instructions/ Master control relay instructions	BAS, BAF
Timer instructions	SP, SE, SD, SS/SSU, SF/SFD, FR, SEC	Conversion instructions	CFW, CSW, CSD DEF, DED, DUF, DUD, GFD, FDG	Stop commands	STP, STS, STW
Counter instructions	CU/SSU, CD/SFD, FR, SEC	Word logic instructions	AW, OW, XOW	Processing functions	DO <Formal parameter>
Load and transfer instructions	L, LD, LW, LDW, TL PB, L QB, L PW, L QW, T PB, T QB, T PW, T QW	Shift and rotate instructions	SLW, SLD, SRW, SRD, SVW, SVD, RLD, RRD		DO FW, DO DW
(continued on next page)	LY GB / GW / GD / CB / CW / CD, LW GW / GD / CW / CD, TY GB / GW / GD / CB / CW / CD, TW GW / GD / CW / CD	Data block instructions	G, CX G, GX	Absolute memory addressing	LIR, TIR, LDI, TDI
Page commands	ACR, TSC, TSG	(continued on next page)		Block transfers	TNB, TNW, TXB, TXW
Math functions	-	Null instructions	BLD xxx NOP 0, NOP 1	Interrupt commands	LIM, SIM, IAE, RAE, IA, RA
				(continued on next page)	

Fonte: SIEMENS

ANEXO C – Características do microcontrolador AT89C51ED2

Features

- 80C52 Compatible
 - 8051 Instruction Compatible
 - Six 8-bit I/O Ports (64 Pins or 68 Pins Versions)
 - Four 8-bit I/O Ports (44 Pins Version)
 - Three 16-bit Timer/Counters
 - 256 Bytes Scratch Pad RAM
 - 9 Interrupt Sources with 4 Priority Levels
- Integrated Power Monitor (POR/PFD) to Supervise Internal Power Supply
- ISP (In-System Programming) Using Standard V_{CC} Power Supply
- 2048 Bytes Boot ROM Contains Low Level Flash Programming Routines and a Default Serial Loader
- High-speed Architecture
 - In Standard Mode:
 - 40 MHz (V_{CC} 2.7V to 5.5V, both Internal and external code execution)
 - 60 MHz (V_{CC} 4.5V to 5.5V and Internal Code execution only)
 - In X2 mode (6 Clocks/machine cycle)
 - 20 MHz (V_{CC} 2.7V to 5.5V, both Internal and external code execution)
 - 30 MHz (V_{CC} 4.5V to 5.5V and Internal Code execution only)
- 64K Bytes On-chip Flash Program/Data Memory
 - Byte and Page (128 Bytes) Erase and Write
 - 100k Write Cycles
- On-chip 1792 bytes Expanded RAM (XRAM)
 - Software Selectable Size (0, 256, 512, 768, 1024, 1792 Bytes)
 - 768 Bytes Selected at Reset for T89C51RD2 Compatibility
- On-chip 2048 Bytes EEPROM Block for Data Storage (AT89C51ED2 Only)
 - 100K Write Cycles
- Dual Data Pointer
- Variable Length MOVX for Slow RAM/Peripherals
- Improved X2 Mode with Independent Selection for CPU and Each Peripheral
- Keyboard Interrupt Interface on Port 1
- SPI Interface (Master/Slave Mode)
- 8-bit Clock Prescaler
- 16-bit Programmable Counter Array
 - High Speed Output
 - Compare/Capture
 - Pulse Width Modulator
 - Watchdog Timer Capabilities
- Asynchronous Port Reset
- Full-duplex Enhanced UART with Dedicated Internal Baud Rate Generator
- Low EMI (Inhibit ALE)
- Hardware Watchdog Timer (One-time Enabled with Reset-Out), Power-off Flag
- Power Control Modes: Idle Mode, Power-down Mode
- Single Range Power Supply: 2.7V to 5.5V
- Industrial Temperature Range (-40 to +85°C)



8-bit Flash Microcontroller

AT89C51RD2
AT89C51ED2

Fonte: ATMEL